

RV32I 5-Stage Pipelined CPU — Design Defense Reference

Every decision, every port, every number, and where it comes from

Computer Organization · BIT Y4T1 · Kantaphon

2026-09-16

Contents

How to use this document	2
0. Fact sheet — the numbers, and where each one comes from	3
1. The ISA decision	4
1.1 Why RISC-V rather than MIPS, x86 or ARM	4
1.2 Why 46 encodings — 37 core + 9 system	5
1.3 Why this exact subset — what is in, what is out, and why	5
1.4 The six instruction formats, and why each is laid out the way it is	6
2. The pipeline	7
2.1 Why five stages	7
2.2 What each stage does	8
2.3 Why in-order issue and in-order completion	9
2.4 Why Harvard (separate IMEM and DMEM)	9
2.5 What each pipeline register carries, and why	9
2.6 The PC-select mux and its priority order	12
3. Module reference — all 19 modules, every port	12
3.1 pc.v — the program counter	13
3.2 imem.v — instruction memory	13
3.3 bht.v — 64-entry branch history table	14
3.4 if_id.v — IF/ID pipeline register	15
3.5 control.v — the main decoder	15
3.6 alu_ctrl.v — second-level ALU decode	17
3.7 imm_gen.v — immediate generator	17
3.8 regfile.v — 32 × 32 register file	18
3.9 id_ex.v — ID/EX pipeline register	19
3.10 forward_unit.v — EX operand bypass selects	20
3.11 alu.v — the arithmetic/logic unit	21
3.12 branch_unit.v — branch condition evaluation	22
3.13 csr.v — CSR file, trap entry, and mret	22
3.14 ex_mem.v — EX/MEM pipeline register	24
3.15 dmem.v — data memory with byte lanes	24
3.16 mem_wb.v — MEM/WB pipeline register	25
3.17 hazard_unit.v — interlock and flush generation	26
3.18 perf_counters.v — performance counters	27
3.19 cpu_top.v — the top level	27
4. Why N bits — every width decision	29
4.1 Why a 32-bit datapath	29
4.2 Why 32 registers, and why x0 is hardwired to zero	29

4.3	The immediate formats, bit by bit	30
4.4	Why 5-bit shift amounts	31
4.5	Word-aligned PC, byte-addressed memory	31
4.6	Why 4 KB IMEM and 4 KB DMEM	31
4.7	BHT sizing: why 64 entries, 2 bits, and <code>pc[7:2]</code>	32
4.8	CSR bit fields — why those bits	32
5.	The hazard scheme	33
5.1	Why forward from EX/MEM and MEM/WB, and only those	33
5.2	Why EX/MEM has priority over MEM/WB	34
5.3	Why a load-use hazard needs a stall, and why exactly one cycle	34
5.4	Why <code>jal</code> costs 1 bubble but branch and <code>jalr</code> cost 2	35
5.5	Why store data (<code>rs2</code>) is forwarded	35
5.6	Why the WB→ID internal bypass exists	36
5.7	The interaction that had to be fixed: ID-stage redirects during a stall	36
5.8	The <code>FORWARDING = 0</code> build — how the comparison is made fair	36
6.	Performance and resources — every number, and why it is that number	37
6.1	CPI: forwarding on versus off	37
6.2	Resources: 1461 LUTs, 819 flip-flops, 2 BRAMs	38
6.3	What maps to what — and the <code>ram_style</code> story	38
6.4	Why 74 MHz, and what the critical path actually is	39
6.5	Two methodological points about the synthesis numbers	40
7.	The bonus items — rationale and results	41
7.1	Bonus A — the 2-bit branch history table	41
7.2	Bonus B — interrupts, CSRs and <code>mret</code>	44
7.3	Bonus C — the instruction cache: designed, not built	46
8.	Verification methodology — and why each choice was made	46
8.1	Why write our own assembler and golden ISS	46
8.2	Why the unit-test vectors are machine-generated, not hand-written	47
8.3	Why trace-diff instead of a final-state check	47
8.4	The hard case: aligning an interrupted program against the reference model	48
8.5	Why mutation testing — showing the testbench fails on a broken DUT	48
8.6	Why test at all four <code>FORWARDING</code> × <code>BHT_ENABLE</code> combinations	49
8.7	The full test inventory	49
9.	Anticipated examiner questions and answers	50
9.1	ISA and encoding	50
9.2	Pipeline and datapath	51
9.3	Hazards	52
9.4	Memory	53
9.5	CSRs, traps and interrupts	53
9.6	Branch prediction	54
9.7	Verification	55
9.8	Synthesis and timing	56
9.9	Scope, process and the awkward questions	57
10.	Known limitations — the honest list	57
	Appendix A — where every claim comes from	58

How to use this document

This is the oral-exam reference for the RV32I pipelined CPU. It is written to answer “why?” for every decision in the design, and “what does this do?” for every module and every port. It is not a summary — the summary is

docs/DESIGN.md. This is the deeper layer underneath it.

Provenance rule used throughout: every number in this document is either (a) read out of a file in the repository, or (b) derived arithmetically from numbers that were, with the derivation shown. Each such claim names its source inline — a docs/DESIGN.md section, a vivado/reports/*.txt file, or an RTL file and its logic. If a number has no source named next to it, it is a definition, not a measurement.

Reading order for revision:

1. §0 — the fact sheet. Every number you could be asked for, on one page.
2. §9 — the anticipated Q&A. Read this last before the exam; it is §1–§8 compressed into answers.
3. §3 — the module reference. Use it to look up a specific port when asked “what is this signal for?”

0. Fact sheet — the numbers, and where each one comes from

Quantity	Value	Source
ISA	RISC-V RV32I + M-mode CSR subset	PROJECT-REQUIREMENTS.md §2
Encodings implemented	46 = 37 core + 9 system	DESIGN.md §1.2
XLEN / register count	32 bits / 32 registers, x0 hardwired 0	DESIGN.md §1.1
Pipeline	5 stages, in-order issue, in-order completion	DESIGN.md §1.1
RTL modules	19	rtl/
IMEM / DMEM	4 KB each = 1024×32 bit, Harvard, single-cycle sync	DESIGN.md §7
Branch resolution	jal in ID (1 bubble); branch/jalr in EX (2 bubbles)	DESIGN.md §6.3
Load-use penalty	exactly 1 stall cycle	DESIGN.md §6.2
BHT	64 entries $\times$ 2-bit saturating, index pc[7:2], reset 01	rtl/bht.v
CSRs implemented	mstatus(MIE,MPIE), mie(MEIE), mtvec, mepc, mcause	rtl/csr.v
CPI, forwarding on (fib+bsort+bloop)	1.387 (6799 cycles / 4902 insns)	DESIGN.md §11.2
CPI, forwarding off (same three)	1.837 (9003 cycles / 4902 insns)	DESIGN.md §11.2
Forwarding speedup	1.32$\times$ overall; 1.65 $\times$ on fib, 1.22 $\times$ on bloop	DESIGN.md §11.2
BHT best case (bpred)	95.5 % accuracy, −1296 cycles (−14.5 %)	DESIGN.md §11.3
BHT worst case (bloop)	49.9 % accuracy, +400 cycles (+10.0 %)	DESIGN.md §11.3
Target part	xc7a35tcpg236-1, Artix-7, speed grade −1	vivado/reports/utilization.txt
LUTs (full design @10 ns)	1461 = 1417 logic + 44 distributed RAM	utilization.txt §1
Flip-flops (full design)	819 (755 with reset, 64 with set)	utilization.txt §1, §1.1
Block RAMs	2 $\times$ RAMB36E1 (IMEM, DMEM)	utilization.txt §3, memory.txt
Base machine (BHT_ENABLE=0)	1245 LUT, 689 FF, 2 BRAM	utilization_base.txt
Cost of the BHT	+216 LUT, +130 FF	1461–1245, 819–689
fmax achieved (setup met)	74.07 MHz at 13.5 ns, WNS +0.052 ns, 0 failing	timing_13.5ns.txt
100 MHz result	not met: WNS −3.235 ns, 555/2956 failing endpoints	timing.txt

Quantity	Value	Source
Critical path	12.982 ns, 15 logic levels, 35 % logic / 65 % route	timing.txt
Critical path start / end	u_dmem/mem_reg/CLKBWRCLK → u_pc/pc_q_reg[31]/CE	timing.txt
Largest single term	BRAM CLKBWRCLK→DOBDO = 2.454 ns (19 % of path)	timing.txt
Hold (all paths)	WHS −0.120 ns, 483 endpoints, all from the rst port	hold.txt
Hold (register→register)	MET +0.071 ns @10 ns, +0.112 @13.5 ns, +0.119 base	hold_reg2reg*.txt
Toolchain self-check	4323 assembler/ISS cross-checks	DESIGN.md §10.2
Unit-test vectors	1865 + 5072 + 76 + 667 + 16072 + 32 + 10 + 12 + 19 + 2600	DESIGN.md §10.3, §10.10
Program tests	46 per-instruction + 9 hazard + 5 program + 7 bring-up	DESIGN.md §10.10
Test matrix	all suites at 4 × (FORWARDING × BHT_ENABLE) combinations	DESIGN.md §10.10

The five numbers most likely to be asked for, with the one-line defense attached:

- **46 instructions** — the brief asked for more than 16; this is 2.9× that, and it is a complete architectural ISA subset rather than an arbitrary selection.
- **CPI 1.39 vs 1.84** — this is the “show the performance of your CPU” deliverable: the same machine, the same programs, one compile-time parameter changed.
- **74 MHz** — the frequency at which a place-and-route run actually closed timing, not a frequency extrapolated from a failing run.
- **1461 LUT / 819 FF / 2 BRAM** — post-place-and-route, out-of-context, with the memories verified to have landed in the primitives they were supposed to land in.
- **0 differing trace lines** — every program’s commit trace is byte-identical to an independently written reference simulator’s.

1. The ISA decision

1.1 Why RISC-V rather than MIPS, x86 or ARM

The course brief says “MIPS/RISC-V or other instruction subsets with more than 16 instructions”, so all of these were admissible. The decision (PROJECT-REQUIREMENTS.md §2, made 2026-09-06) was RV32I, for reasons that are about *verifiability* as much as about the ISA itself.

Versus MIPS. MIPS is the traditional teaching ISA and the course’s named simulator (Mars4_5) is a MIPS tool, so this was the default choice and had to be argued against. Three reasons it was rejected:

1. *Delay slots.* Classic MIPS has an architecturally visible branch delay slot — the instruction after a branch always executes. That is a hazard-hiding hack baked into the ISA: it makes the control-hazard discussion, which is a third of the point of building a pipeline, partly disappear into the instruction set instead of being solved in hardware. RISC-V has no delay slot, so every control hazard has to be handled by flushing, and the flush logic is visible and measurable.
2. *Encoding irregularity.* MIPS immediate handling is less uniform, and its lui/ori constant-building idiom, HI/LO registers for multiply, and coprocessor-0 state are extra surface area with no pedagogical payoff for a 5-stage pipeline.
3. *Encoding regularity in the other direction.* RV32I keeps rs1, rs2 and rd in fixed bit positions across every format that uses them (see §1.4). This is the single property that makes the decode stage cheap: register reads can start before the instruction is fully decoded, because the register addresses are at fixed wires.

Versus x86. x86 is variable-length (1–15 bytes), has complex addressing modes, microcoded instructions, and a decoder that is itself a multi-stage pipeline. Building a 5-stage x86 is not a scaled-down version of this project; it is a different and much larger project whose difficulty lies entirely in decode rather than in pipelining. It would also have no usable reference model we could write ourselves.

Versus ARM. ARM (A32) is a reasonable pipelining target, but two things rule it out for a course project: conditional execution on nearly every instruction (predication) complicates the write-back and flush logic without teaching anything about hazards that forwarding does not, and the ISA is proprietary — the specification is not freely redistributable in the way the RISC-V specs are, which matters for a document that has to cite the encoding tables.

The decisive argument: we needed to be able to build our own oracle. The course names Mars4_5 as the simulation tool. Mars executes MIPS and cannot assemble or simulate RISC-V at all, so choosing RISC-V meant the project had no ready-made reference. Rather than treat that as a cost, it was treated as a requirement: `tools/asm.py` (assembler) and `tools/iss.py` (golden instruction-set simulator) were written from the specification. RV32I is small enough and regular enough that writing a correct reference model in Python is a day’s work, which is exactly what makes the differential testing of §8 possible. The same would not have been true for ARM or x86.

1.2 Why 46 encodings — 37 core + 9 system

The brief’s floor is “more than 16 instructions”. The design implements 46:

Group	Count	Instructions
R-type	10	add sub sll slt sltu xor srl sra or and
I-arithmetic	9	addi slti sltiu xori ori andi slli srli srai
Loads	5	lb lh lw lbu lhu
Stores	3	sb sh sw
Branches	6	beq bne blt bge bltu bgeu
U-type	2	lui auipc
Jumps	2	jal jalr
System	9	ecall ebreak csrrw csrrs csrrc csrrwi csrrsi csrrci mret

(Source: DESIGN.md §1.2. $10+9+5+3+6+2+2 = 37$ core, $+9$ system = 46.)

The count is stated as “46 encodings (37 core + 9 system)” *consistently everywhere* — the README, DESIGN.md, the proposal and this document — because “how many instructions” has more than one defensible answer and an inconsistent count is the easiest thing for an examiner to catch. Three counting conventions exist and are worth knowing:

- **46** — every distinct encoding the decoder recognises. This is the number used.
- **37** — the core user-level instruction count, excluding the system group.
- **40** — base RV32I as the unprivileged specification enumerates it. That 40 is exactly our 37 core instructions plus `fence`, `ecall` and `ebreak`. We implement **39 of those 40**; the only base instruction omitted is `fence`. On top of the base we add 7 more: the six Zicsr instructions and `mret` from the privileged specification. $39 + 7 = 46$.

So the precise claim, and the one to make if pressed, is: “*the complete RV32I base integer set except `fence`, plus `Zicsr` and `mret`*”. `fence.i` is not part of the base set at all — it lives in the separate Zifencei extension — so omitting it is not a gap in RV32I coverage. Why `fence` itself is omitted rather than implemented as the NOP it would architecturally be argued in §1.3.

1.3 Why this exact subset — what is in, what is out, and why

`ecall` is in because it is the only way to demonstrate a *synchronous* trap, and the synchronous/asynchronous distinction is the interesting half of the interrupt bonus: the `mepc` handling differs between the two cases (§7.2),

and that asymmetry is the classic trap bug. Without `ecall` there is only one trap path and the asymmetry cannot be shown.

ebreak is in because RV32I has no halt instruction and simulation needs a defined end. `ebreak` retires normally, is counted and traced like any other instruction, and then raises a sticky `done` flag that freezes the pipeline (`cpu_top.v`, `done_r`). That gives every test program an exact, architecturally-defined stopping point: the trace ends at the `ebreak` line and nothing behind it commits. The alternative — running for a fixed number of cycles and then inspecting state — would make the commit-trace diff impossible to align.

fence and fence.i are out. `fence` orders memory operations as seen by *other* harts or by DMA. This machine is single-hart with no coherent agents, no store buffer, no write-back cache and in-order single-cycle memory, so every memory operation is already globally ordered by construction: `fence` would be architecturally required to do nothing. `fence.i` synchronises the instruction stream with data writes — it matters when a program writes its own instructions, but IMEM here is a read-only ROM initialised by `$readmemh` and there is no store path into it, so self-modifying code is impossible. Implementing either would be implementing a NOP and calling it an instruction. They decode as illegal instead, which is an honest statement of what the machine supports.

wfi is out, and is explicitly listed as illegal in `control.v`'s SYSTEM decode (`csr_addr 0x105`). `wfi` means “stall the core until an interrupt arrives”. It would require a sleep state in the fetch unit and a wake path from the interrupt logic — real logic, real new states, and a new class of thing to verify (what retires when the core wakes?). The interrupt demonstration does not need it: the interrupt is taken against a running instruction stream, which is the harder and more interesting case anyway.

The CSR op subset. All six CSR instructions are implemented (`csrrw`, `csrrs`, `csrrc` and their immediate forms). This is *not* a subset — it is the complete Zicsr set. What *is* subsetting is the CSR *registers*: five are implemented (`mstatus`, `mie`, `mtvec`, `mepc`, `mcause`) and the rest read zero and ignore writes. Justification in §7.2. The reason to implement all six *instructions* even though the demonstration ISR only needs `csrrw/csrrs` is that they cost almost nothing — `csr.v`'s read-modify-write is a three-way mux (`src, old|src, old&~src`) and the immediate forms only change where `src` comes from — while omitting them would leave a jagged edge in the decode table that a reader would have to be told about.

Multiply/divide (the M extension) is out. It is a separate extension, not part of RV32I. A single-cycle multiplier would either blow the critical path or need a multi-cycle stall path through EX — a structural hazard class this design does not otherwise have. The test programs (`fib`, `bsort`, `bloop`, `bpred`) are written to need only RV32I.

1.4 The six instruction formats, and why each is laid out the way it is

Format	31:25	24:20	19:15	14:12	11:7	6:0	Used by
R	funct7	rs2	rs1	funct3	rd	opcode	R-type
I	imm[11:0]		rs1	funct3	rd	opcode	I-arith, loads, jalr, CSR
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	stores
B	imm[12 10:5]	rs2	rs1	funct3	imm[4:1 11]	opcode	branches
U	imm[31:12]				rd	opcode	lui, auipc
J	imm[20 10:1 19:12]				rd	opcode	jal

(Source: `DESIGN.md` §1.3; implemented in `rtl/imm_gen.v`.)

Three design principles are visible in this table, and an examiner asking “why is the B-type immediate scrambled?” is asking about the third one.

Principle 1 — fixed register positions. `rs1` is always `inst[19:15]`, `rs2` is always `inst[24:20]`, `rd` is always `inst[11:7]`, in every format that has those fields. Nothing has to be decoded before the register file can be

read. In this design `cpu_top.v` wires `id_rs1 = id_inst[19:15]` and `id_rs2 = id_inst[24:20]` straight off the instruction word and into `regfile`'s read address ports, in parallel with `control.v` decoding the opcode. If the positions moved between formats, the register read would have to wait for the decoder and ID would be two levels of logic deeper.

Principle 2 — the sign bit never moves. Every signed immediate takes its sign from `inst[31]`. Look at `imm_gen.v`: the I, S, B and J cases all begin `{n{inst[31]}}, ...`. The sign-extension logic is therefore one fan-out from a single wire, identical in all four cases, rather than a mux over four different sign-bit positions.

Principle 3 — every immediate bit keeps the same physical wire position across formats. This is what the “scrambling” of B and J buys. Compare S and B: S is `imm[11:5]` in `inst[31:25]` and `imm[4:0]` in `inst[11:7]`. B is *almost* the same wiring — `imm[10:5]` in `inst[30:25]` and `imm[4:1]` in `inst[11:8]` — with only bits 11 and 12 displaced, because the B immediate has an implicit zero in bit 0 and so shifts everything up by one. Rather than re-route every bit, the format keeps bits 10:1 in the same physical wires as S's bits 11:2 and parks the two odd bits (11 and 12) in the two slots that freed up. The same is true of U versus J.

The payoff is that `imm_gen.v` is *pure wiring* — no shifters, no adders, no barrel logic, just a 6-way mux over sign-extension patterns of fixed wire bundles. The RTL proves it: the entire module is one `case` statement of concatenations (`rtl/imm_gen.v` lines 40–55). A format design that put the immediate in “natural” bit order would need a different mux per format and more routing, which is a real area and delay cost in the ID stage.

Why the B-type immediate's bit 0 is not encoded. All RV32I instructions are 4-byte aligned, so a branch target is always even; bit 0 of the offset is therefore always zero and encoding it would waste an encoding bit. Dropping it and hard-wiring `1'b0` in the generator (`imm_gen.v`, the trailing `1'b0` in the B and J cases) doubles the reachable branch range for free: a 12-bit encoded field covers ± 4 KiB instead of ± 2 KiB. The same argument applies to J: 20 encoded bits, `1'b0` appended, giving ± 1 MiB.

Why U-type has no sign extension. `lui/auipc` place `inst[31:12]` directly into bits `[31:12]` of the result and zero the low 12 (`imm_gen.v`: `{inst[31:12], 12'b0}`). There is nothing above bit 31 to extend *into* — the immediate already reaches the top of the word. `inst[31]` is still the value's sign bit, it is simply used in place rather than replicated. This is why the `lui+addi` constant-building idiom needs care: the `addi`'s immediate is sign-extended, so building `0xFFFFF800` as `lui+addi` requires the assembler to add 1 to the upper part when the lower part is negative. `tools/asm.py` does exactly that in its `li` expansion, and `tools/test_tools.py` checks it at the boundary values (DESIGN.md §10.2).

The sixth immediate: `zimm`. The CSR immediate forms (`csrrwi/csrrsi/csrrci`) take a 5-bit unsigned immediate from `inst[19:15]` — the field that is `rs1` in every other I-type instruction. It is **zero-extended**, the only zero-extended immediate in the design (`imm_gen.v`, `IMM_Z: imm = {27'b0, inst[19:15]}`). It reuses the `rs1` field because those instructions do not read a register — that is exactly why `cpu_top.v`'s `id_uses_rs1` excludes them (`id_csr_en && !id_csr_imm`), so the hazard unit does not invent a dependency on a register number that is really an immediate (§5.3).

2. The pipeline

2.1 Why five stages

Why not single-cycle? A single-cycle machine has $CPI = 1.0$ by definition, which sounds better than our 1.39, but its clock period must cover the *entire* worst-case instruction path in one cycle: instruction fetch + register read + ALU + data memory + register write-back. In this design's own measured terms (`timing.txt`), the DMEM read alone is 2.454 ns of clock-to-output before any logic, and the paths on either side of it are another ~10 ns. A single-cycle version would be looking at something in the region of 25–30 ns per instruction — roughly 35 MHz — against a pipelined 13.5 ns clock running at CPI 1.39. The pipelined machine retires an instruction every $13.5 \times 1.39 = 18.8$ ns on average, versus roughly 28 ns, and the gap widens with every stage the critical path gets split across. More importantly for a *course* project: a single-cycle machine has no hazards at all, so there is nothing to demonstrate. Forwarding, stalling and flushing — the entire subject of the exercise — exist only because of pipelining.

Why not deeper, e.g. 8 stages? A deeper pipeline shortens the critical path further and raises fmax, but it costs on three axes, none of which buys anything new here:

1. *Longer hazard distances.* With more stages between the producer and the consumer, the forwarding network grows — an 8-stage machine with the same in-order structure needs bypasses from three or four stages instead of two, which is more multiplexing in EX, i.e. more delay in the very stage the extra depth was supposed to relieve.
2. *Deeper branch shadows.* Resolving a branch at stage 5 of 8 instead of stage 3 of 5 means four killed instructions per taken branch instead of two. At **bloop**'s branch density the flush penalty would swamp the frequency gain, and you then *need* a good branch predictor rather than being able to demonstrate one as a bonus.
3. *No new concepts.* The three hazard classes (structural, data, control) are all already present and all already demonstrable at five stages. A deeper pipeline adds quantity, not kind.

Five stages is the shallowest pipeline that still exposes all three hazard classes. That is the argument, and it is the one recorded in the proposal (**Report/midterm-proposal.md** §2): *“the shallowest design that still exposes all three hazard classes, which is the point of the exercise.”*

Why this particular split? Because it matches the natural data dependencies of a load/store architecture: an address cannot be computed until the operands are read, and memory cannot be accessed until the address exists, and the result cannot be written back until memory has answered. IF | ID | EX | MEM | WB is that dependency chain cut at its four natural joints.

2.2 What each stage does

Stage	Modules involved	Work performed
IF	<code>pc.v</code> , <code>imem.v</code> , <code>bht.v</code> , <code>if_id.v</code>	Present <code>pc_q</code> to IMEM; look up the BHT with the same PC; choose the next PC through the PC-select mux; latch PC, valid and the 2-bit prediction into IF/ID
ID	<code>control.v</code> , <code>alu_ctrl.v</code> , <code>imm_gen.v</code> , <code>regfile.v</code>	Decode the instruction word into the full control word; generate the immediate; read <code>rs1/rs2</code> (with the WB→ID bypass); compute and take the <code>jal</code> / predicted-branch redirect; supply the hazard unit its comparison inputs
EX	<code>forward_unit.v</code> , <code>alu.v</code> , <code>branch_unit.v</code> , <code>csr.v</code>	Select each of three operands through the forwarding muxes; compute the ALU result; evaluate the branch condition; compute branch and <code>jalr</code> targets; perform the CSR read-modify-write; take traps and <code>mret</code> ; update the BHT; select <code>res</code> from {ALU, PC+4, CSR}
MEM	<code>dmem.v</code>	Byte-lane store or synchronous load; the memory word and the address low bits are registered on this edge
WB	(<code>cpu_top.v</code> mux)	Choose between <code>res</code> and the load data; write the register file (suppressed for x0); drive the commit-trace port; pulse <code>retire</code>

Two placements in this table are choices worth defending, because a textbook diagram would put them elsewhere:

The result mux is in EX, not WB. `cpu_top.v` selects `ex_res` from {`alu_y`, `ex_pc+4`, `csr_rdata`} *before* the EX/MEM register, and WB then only picks between `res` and the load data. The reason is forwarding correctness:

the EX/MEM forwarding source must be the value the instruction is actually going to write back. For a `jal/jalr` that is the link address `PC+4`, not the ALU output (which is the jump *target*); for a `csrrw` it is the old CSR value, not the unused ALU result. If the mux lived in WB, `fwd = 2'b10` would forward a jump target into a dependent instruction. This is written into `ex_mem.v`'s header comment as the module's reason for holding `res` rather than `alu_y`.

Load lane-select and sign extension are in WB, not MEM. `dmem.v` registers the raw memory word, the address low bits and `funct3` on the MEM clock edge, then does the lane select and sign/zero extension combinatorially, so the extended value is valid during WB. The load data is therefore deliberately *not* a MEM/WB field. This saves 32 flip-flops — but it is also the reason steps 2–3 of the critical path sit in the same cycle as the 2.454 ns BRAM read (§6.4), and moving that extension behind the MEM/WB register is listed as the first timing fix (DESIGN.md §11.5).

2.3 Why in-order issue and in-order completion

In-order issue means instructions enter EX in program order. **In-order completion** means they write architectural state in program order. This machine does both, and the consequences are what make it verifiable:

- The commit trace is a total order that matches the program, so it can be compared line-by-line against a simple sequential reference model (§8). An out-of-order machine would need a reorder buffer before it could produce such a trace at all.
- Precise traps come for free. When a trap is taken in EX, everything older is already past EX and will complete; everything younger is in ID and IF and is flushed. There is no in-flight younger instruction that has already modified architectural state, so `mepc` unambiguously identifies the boundary. This is exactly what §7.2's interrupt timing diagram shows, and it is why `tb_irq.v` can assert “at most two instructions retire between the trap and the handler's first instruction”.
- The hazard logic is a fixed, small comparison. With in-order issue, the only possible producers of a value the EX stage needs are the instructions in MEM and WB — two candidates, hence two forwarding paths and a 2-bit select (§5).

Out-of-order execution would raise IPC on dependent code, but it requires register renaming, a scheduler, a reorder buffer and precise-exception machinery — an order of magnitude more logic, and a verification problem that could not be solved with a line-by-line trace diff in the time available.

2.4 Why Harvard (separate IMEM and DMEM)

A single unified memory creates a **structural hazard**: IF wants to read an instruction every cycle, and MEM wants to read or write data in the same cycle for every load/store. One memory with one port cannot serve both, so a von Neumann version of this pipeline would have to stall IF on every load and store — roughly one instruction in four in typical code, and a stall that no amount of forwarding can remove.

Harvard removes the hazard by construction: `imem.v` and `dmem.v` are separate arrays with separate ports, and IF and MEM never contend. Post-synthesis this shows up as two independent `RAMB36E1` primitives (`memory.txt`), which is also the natural fit for an FPGA — the block RAMs are physically separate resources, so “one big memory” would not even have been cheaper.

The honest caveat, which belongs in the answer: this is a *microarchitectural* Harvard split, not an architectural one. Real machines present a unified address space and get the same effect from separate L1 instruction and data caches in front of unified memory. Here the split is visible to software — IMEM address `0x40` and DMEM address `0x40` are different locations — which is acceptable because programs are loaded by `$readmemh` rather than by a loader, and it is documented in DESIGN.md §7 (“Both memories are addressed `0x00000000–0x00000FFF` in their own space”). The consequence is that this machine cannot execute code it generates at run time, which is the same restriction the absence of `fence.i` implies (§1.3) — the two decisions are consistent with each other.

2.5 What each pipeline register carries, and why

Every stage boundary is an explicit module (`if_id.v`, `id_ex.v`, `ex_mem.v`, `mem_wb.v`), and all four have the identical four control inputs: `clk`, `rst`, `stall` (hold), `flush` (overwrite with a bubble). That uniformity is the reason the

hazard logic of §5 could be added in a later build step without touching the datapath — the ports were already there, driven with constants.

The bubble encoding is all-zeros. In every one of the four registers, a flush writes {PW{1'b0}} across the whole payload. That is not an arbitrary poison value: with every control bit zero (`reg_we = mem_re = mem_we = csr_en = csr_we = branch = jalr = mret = ecall = ebreak = 0`) and `valid = 0`, a flushed slot is a genuine architectural NOP that cannot write a register, cannot touch memory, cannot redirect the PC, is not counted by `retire`, and does not appear in the commit trace. Reset uses the same encoding, so “after reset” and “just flushed” are the same state and there is one case to reason about instead of two.

flush has priority over stall in all four registers. The code shape is `if (rst || flush) ... else if (!stall) ...` — see any of the four modules. The reason is in §5.4: a redirect makes any stall the flushed instruction raised moot, and honouring the stall instead would deadlock the redirect.

IF/ID (`if_id.v`)

Field	Width	Why it is here
<code>pc_q</code>	32	Base for the <code>jal</code> /predicted-branch target (<code>if_id_pc + id_imm</code>), the future <code>mepc</code> , and the trace
<code>valid_q</code>	1	0 = bubble; ID substitutes a NOP for the instruction word
<code>pred_state_q</code>	2	The BHT counter state looked up in IF, carried so ID needs no second table read

The instruction word is deliberately not in this register. `imem.v` is a synchronous-read memory, so its own output register *is* the instruction half of the IF/ID boundary. `cpu_top.v` clocks both with the same enable (`fetch_en`), and ID substitutes `NOP_INST` (0x00000013) whenever `valid_q = 0`. Registering the instruction a second time would add a wasted cycle of fetch latency for no benefit. This is the one place where the design departs from the textbook block diagram, and it is the kind of thing an examiner may notice from the RTL — the answer is “the instruction *is* registered, by the memory, and `if_id.v` carries the metadata that has to stay in step with it”.

A bubble carries `pred_state = 2'b00` (strongly not taken), which is the safe answer for a slot holding no branch.

ID/EX (`id_ex.v`) — 212 payload bits

Grouped by what needs them:

Group	Fields	Purpose
Bookkeeping	<code>valid</code> , <code>illegal</code> , <code>pc</code> , <code>inst</code>	Retire/trace qualification; <code>pc</code> is also the PC+4 link source, the branch-target base and <code>mepc</code> on a trap
Register addresses	<code>rs1_addr</code> , <code>rs2_addr</code> , <code>rd_addr</code>	The forwarding comparisons happen in EX, so the <i>addresses</i> must travel with the instruction, not just the values
Data	<code>rs1_val</code> , <code>rs2_val</code> , <code>imm</code>	Pre-forwarding register values and the selected immediate (also the CSR <code>zimm</code>)
EX control	<code>alu_op</code> , <code>alu_src_a</code> , <code>alu_src_b</code> , <code>funct3</code> , <code>branch</code> , <code>jalr</code> , <code>pred_taken</code>	ALU operation and operand selects; branch condition code; whether ID acted on a taken prediction
MEM control	<code>mem_re</code> , <code>mem_we</code>	Passed through EX untouched

Group	Fields	Purpose
WB control	<code>reg_we</code> , <code>wb_sel</code>	Passed through EX and MEM untouched
CSR/system	<code>csr_en</code> , <code>csr_we</code> , <code>csr_imm</code> , <code>csr_addr</code> , <code>mret</code> , <code>ecall</code> , <code>ebreak</code>	Consumed in EX (CSR RMW, trap entry, <code>mret</code>) except <code>ebreak</code> , which travels to WB

The pattern to notice — and a good thing to say out loud in a defense — is that **control signals are decoded once, in ID, and then travel with the instruction**. EX does not re-decode; MEM does not re-decode. That is why `control.v` is a single combinational block with no state, and why the decode truth table (DESIGN.md §3) is a complete specification of the machine's behaviour: nothing downstream can override it.

`pc` is carried rather than recomputed because three different consumers need it (link value, branch base, `mepc`) at three different times, and because the commit trace needs to print it.

EX/MEM (`ex_mem.v`) — 144 payload bits

Field	Width	Purpose
<code>valid</code> , <code>illegal</code>	1, 1	Retire/trace qualification
<code>pc</code> , <code>inst</code>	32, 32	Commit trace only
<code>rd_addr</code> , <code>reg_we</code>	5, 1	Write-back address, and the forwarding unit's EX/MEM match test
<code>wb_sel</code>	2	Selects load data vs <code>res</code> in WB
<code>res</code>	32	The already-selected result: ALU output, or <code>PC+4</code> , or the old CSR value; for a load/store it is the effective byte address
<code>store_data</code>	32	Forwarded <code>rs2</code> for <code>sb/sh/sw</code>
<code>funct3</code>	3	Load/store width and signedness, consumed by <code>dmem.v</code>
<code>mem_re</code> , <code>mem_we</code>	1, 1	Drive <code>dmem.v</code>
<code>ebreak</code>	1	Sets the sticky halt when it retires

Note that `res` is doing double duty: result value *and* memory address. They are never both needed — a load or store has no other result, and an ALU instruction has no address — so carrying one 32-bit field instead of two saves 32 flip-flops with no ambiguity. The commit trace exploits the same overlap: `trace_mem_addr` is `MEM/WB.res`.

MEM/WB (`mem_wb.v`) — 140 payload bits

Field	Width	Purpose
<code>valid</code> , <code>illegal</code>	1, 1	Qualify <code>retire</code> and the trace
<code>pc</code> , <code>inst</code>	32, 32	Commit trace
<code>rd_addr</code> , <code>reg_we</code>	5, 1	Register-file write port, and the MEM/WB forwarding match
<code>wb_sel</code>	2	Write-back mux select
<code>res</code>	32	Non-memory result; also the trace's store address
<code>mem_we</code>	1	A store retired (trace only)
<code>mem_val</code>	32	Store data masked to width, captured from <code>dmem.v</code> at MEM (trace only)
<code>ebreak</code>	1	Sticky halt

Field	Width	Purpose
-------	-------	---------

`mem_val` and the `pc/inst` pair are pure observation fields — they have no datapath role and exist so the testbench can compare a commit trace against the reference model. On a real product they would not be synthesised; here they are, and `constraints.xdc` declares the resulting 361 output bits as false paths so they cannot distort the reported critical path (§6.5).

2.6 The PC-select mux and its priority order

`cpu_top.v` implements it as a priority chain:

```

if (trap_taken)    pc_next = mtvec_val;      // EX
else if (mret_taken) pc_next = mepc_val;     // EX
else if (br_redirect) pc_next = ex_correct_target; // EX
else if (jalr_taken) pc_next = ex_jalr_target; // EX
else if (jal_taken)  pc_next = id_redirect_target; // ID
else if (bht_taken)  pc_next = id_redirect_target; // ID
else                pc_next = pc_q + 32'd4;

```

Three rules are encoded in that order, and each is worth being able to state:

1. **Everything resolved in EX outranks anything resolved in ID or IF.** The EX instruction is *older*. An ID-stage `jal` that loses to an EX-stage branch is itself on the wrong path and is about to be flushed, so honouring its redirect would be following a dead instruction.
2. **Trap and mret outrank the branch and jalr targets.** They are properties of the same EX instruction, and a trapping branch must go to the handler rather than to its own target. `trap_taken` also gates `br_redirect` directly (`ex_branch_valid = ex_valid & ex_branch & ~trap_taken`) so the two cannot fight.
3. **A redirect overrides a stall.** `fetch_en = ~halt & (redirect | ~stall)` — the PC and the instruction memory advance to the redirect target even on a stalled cycle, because the stalling instruction is being killed anyway. See §5.4 for the subtlety this creates for ID-stage redirects, which is one of the more interesting bugs the design had to solve.

`reset` is not in the mux: it is handled inside `pc.v` (`if (rst) pc_q <= 32'b0`), which is a synchronous, active-high reset to address zero.

3. Module reference — all 19 modules, every port

This section is the “what does this part do, what does this port do” reference. Modules are ordered by where they sit in the pipeline. For each: its purpose, its parameters, a complete port table (direction, width, meaning) and the key logic, followed by the points most likely to be probed.

Summary of the 19:

#	Module	Stage	Kind	Lines (incl. comments)
1	<code>pc.v</code>	IF	sequential	19
2	<code>imem.v</code>	IF	memory	30
3	<code>bht.v</code>	IF (lookup) / EX (update)	memory + FSM	104
4	<code>if_id.v</code>	IF→ID	pipeline register	57
5	<code>control.v</code>	ID	combinational	330
6	<code>alu_ctrl.v</code>	ID	combinational	92
7	<code>imm_gen.v</code>	ID	combinational	60
8	<code>regfile.v</code>	ID (read) / WB (write)	memory	37
9	<code>id_ex.v</code>	ID→EX	pipeline register	130
10	<code>forward_unit.v</code>	EX	combinational	97
11	<code>alu.v</code>	EX	combinational	28

#	Module	Stage	Kind	Lines (incl. comments)
12	<code>branch_unit.v</code>	EX	combinational	29
13	<code>csr.v</code>	EX	sequential	168
14	<code>ex_mem.v</code>	EX→MEM	pipeline register	69
15	<code>dmem.v</code>	MEM	memory	111
16	<code>mem_wb.v</code>	MEM→WB	pipeline register	63
17	<code>hazard_unit.v</code>	global	combinational	150
18	<code>perf_counters.v</code>	global	sequential	41
19	<code>cpu_top.v</code>	—	structural	730

3.1 `pc.v` — the program counter

Purpose. Hold the address of the instruction being fetched. It is the only piece of architectural state in IF.

Port	Dir	Width	Meaning
<code>clk</code>	in	1	Clock, single domain, rising edge
<code>rst</code>	in	1	Synchronous, active-high reset → <code>pc_q = 0</code>
<code>en</code>	in	1	Update enable; low holds the current PC (stall or halt)
<code>pc_next</code>	in	32	The next PC chosen by <code>cpu_top</code> 's PC-select mux
<code>pc_q</code>	out	32	Current PC; drives IMEM's address and the BHT lookup

Logic. Three lines: `if (rst) pc_q <= 0; else if (en) pc_q <= pc_next;.`

Defense points.

- *Why is reset synchronous?* FPGA flip-flops have a synchronous reset input that costs nothing; an asynchronous reset needs the dedicated async path and creates a recovery/removal timing check against a signal that may not be synchronised to the clock. The whole design uses synchronous reset consistently — which is why `utilization.txt` §1.1 reports 755 registers with a *synchronous* reset and 64 with a synchronous set, and zero asynchronous ones.
- *Why does `en` exist rather than feeding `pc_q` back through the mux?* Because the enable is also what `imem.v` uses. The fetched instruction lives in IMEM's output register, so holding the PC alone would not hold the instruction — the two have to be frozen by the same signal. `cpu_top.v` drives both from `fetch_en`.
- *Where does `en` come from?* `fetch_en = ~halt & (redirect | ~stall)`. Halt freezes everything; a redirect always re-fetches even during a stall (§5.4).

3.2 `imem.v` — instruction memory

Purpose. 4 KB read-only instruction store, synchronous read, initialised from a hex image produced by `tools/asm.py`.

Parameter. `INIT` — the `$readmemh` file (default `asm/smoke.hex`; synthesis overrides it to `asm/prog/bpred.hex`, see §6.3).

Port	Dir	Width	Meaning
clk	in	1	Clock
en	in	1	Read enable; when low, inst holds its value
addr	in	32	Byte address (the PC); only addr[11:2] is used
inst	out	32	Registered instruction word, valid the cycle <i>after</i> en

Logic. `(* ram_style = "block" *) reg [31:0] mem [0:1023]; initialised by $readmemh, with always @(posedge clk) if (en) inst <= mem[addr[11:2]];`

Defense points.

- *Why is **inst** a register and not combinational?* Because FPGA block RAM is synchronous — a BRAM read *is* a clocked operation. Writing it this way is what lets Vivado infer RAMB36E1 at all. It also means this output register serves as the instruction half of the IF/ID boundary (§2.5), so nothing is lost.
- *Why **addr[11:2]** and not the whole address?* [1:0] are always zero for an aligned instruction fetch, and bits above 11 are outside a 4 KB memory. The address space therefore wraps at 4 KB, which is documented as intentional (DESIGN.md §7).
- *Why the **ram_style** attribute?* Without it Vivado constant-folds a sparsely initialised ROM into LUT logic and can delete it from the netlist entirely. That actually happened and produced a fake 88 MHz result — the full story is in §6.3. The attribute is ignored by xsim, so it changes nothing in simulation.
- *Is it writable?* No. There is no write port, which is why self-modifying code is impossible and **fence.i** is meaningless here (§1.3).

3.3 bht.v — 64-entry branch history table

Purpose. Predict the *direction* of conditional branches. Bonus item A.

Parameter. **ENABLE** — 0 forces “not taken” on every lookup and writes no counter, reproducing the static-not-taken baseline exactly.

Port	Dir	Width	Meaning
clk, rst	in	1, 1	Clock; reset sets all 64 counters to 2'b01
lookup_pc	in	32	The PC being fetched; index is lookup_pc[7:2]
pred_taken	out	1	lookup_state[1] — the prediction (0 when ENABLE = 0)
pred_state	out	2	The full counter state, carried into IF/ID
update_en	in	1	A conditional branch resolved in EX this cycle
update_pc	in	32	EX-stage PC of that branch; index is update_pc[7:2]
update_taken	in	1	Its <i>resolved</i> outcome, from branch_unit

Logic. `reg [1:0] ctr [0:63].` Lookup is combinational (`ctr[lookup_idx]`). Update is synchronous and saturating: **taken** increments unless already 11; **not taken** decrements unless already 00.

Defense points.

- *Why is the lookup combinational and the update clocked?* The lookup has to complete within IF, in parallel with the instruction fetch, so it must not cost a cycle. The update is architectural state and must be a register.
- *What if a lookup and an update hit the same index in the same cycle?* Allowed; the lookup returns the old value. That costs at most one extra mispredict on a branch that recurs faster than the pipeline depth, and it removes a bypass path from the design. `tb_bht.v` tests this case explicitly.
- *Why does the module export `pred_state` (2 bits) rather than just `pred_taken`?* The 2-bit state is carried in IF/ID so that ID can re-derive the prediction without a second table read; only 1 bit (`pred_taken`) then continues into ID/EX for the mispredict comparison.
- *Why no branch target buffer?* The target of a conditional branch is `PC + B-immediate` and ID computes that anyway from the instruction word. A BTB would be a tagged CAM to store something we can recompute for free. The cost of not having one is that a correctly predicted taken branch still costs one bubble rather than zero (§7.1).

3.4 `if_id.v` — IF/ID pipeline register

Purpose. Carry the PC, the valid bit and the BHT prediction across the IF→ID boundary, in step with the instruction word held in IMEM’s output register.

Port	Dir	Width	Meaning
<code>clk, rst</code>	in	1, 1	Clock; reset writes the bubble encoding
<code>stall</code>	in	1	Hold current contents (driven by <code>~fetch_en</code>)
<code>flush</code>	in	1	Overwrite with a bubble; priority over <code>stall</code>
<code>pc_d / pc_q</code>	in/out	32	PC of the instruction being fetched
<code>valid_d / valid_q</code>	in/out	1	<code>valid_d</code> is tied to <code>1'b1</code> ; <code>valid_q = 0</code> means bubble
<code>pred_state_d / _q</code>	in/out	2	BHT counter state looked up with the same PC

Defense points.

- *Why is `valid_d` hardwired to 1?* Because every cycle IF/ID is written with a real fetch — the only way to get `valid_q = 0` is a flush or reset. Tying it to a constant makes that explicit.
- *Why does a bubble carry `pred_state = 00`?* `00` is “strongly not taken”, the safe answer for a slot that contains no branch. It can never cause a spurious redirect.
- *Why is `stall` driven by `~fetch_en` rather than by `stall` directly?* Because `fetch_en` already folds in the halt and redirect-override rules; using it keeps the register, the PC and IMEM under one single condition and removes the possibility of them disagreeing.

3.5 `control.v` — the main decoder

Purpose. Turn the instruction word into the complete control word for the whole pipeline. Purely combinational, no state, decodes all 46 encodings and raises `illegal` for everything else.

Port	Dir	Width	Meaning
<code>opcode</code>	in	7	<code>inst[6:0]</code>

Port	Dir	Width	Meaning
funct3	in	3	<code>inst[14:12]</code>
funct7	in	7	<code>inst[31:25]</code> — also the shift-immediate's upper field
rs1	in	5	<code>inst[19:15]</code> — needed <i>only</i> to suppress the CSR write side
csr_addr	in	12	<code>inst[31:20]</code> — CSR number, and the <code>ecall/ebreak/mret</code> selector
reg_we	out	1	Write <code>rd</code> (the register file separately drops writes to <code>x0</code>)
alu_src_a	out	1	0 = <code>rs1</code> , 1 = PC (asserted only by <code>auipc</code>)
alu_src_b	out	1	0 = <code>rs2</code> , 1 = immediate
imm_sel	out	3	Immediate format: 0 I, 1 S, 2 B, 3 U, 4 J, 5 Z
alu_class	out	2	0 force-ADD, 1 R-type table, 2 I-type table, 3 LUI
mem_re / mem_we	out	1, 1	Data-memory read / write in MEM
wb_sel	out	2	Write-back source: 0 ALU, 1 MEM, 2 PC+4, 3 CSR
branch	out	1	Conditional branch (resolved in EX)
jal / jalr	out	1, 1	<code>jal</code> (resolved in ID) / <code>jalr</code> (resolved in EX)
csr_en	out	1	A CSR instruction: enables the read side
csr_we	out	1	The write side actually occurs (see below)
csr_imm	out	1	Source is <code>zimm</code> rather than <code>rs1</code>
mret / ecall / ebreak	out	1,1,1	The three fixed system encodings
illegal	out	1	Not a recognised encoding

Key logic and the legality rules it enforces (DESIGN.md §3 has the full table):

- **R-type** is legal only with `funct7` = 0x00, or 0x20 with `funct3` = 000 (`sub`) or 101 (`sra`).
- **Shift-immediates** require `inst[31:25]` = 0x00 (`slli`, `srl`) or 0x20 (`srai`).
- **All other I-type instructions ignore funct7 entirely** — those bits are part of the immediate. This is the single most important line in the module.
- Loads accept `funct3` values 000, 001, 010, 100 and 101; stores 000, 001 and 010; branches 000, 001, 100, 101, 110 and 111; `jalr` requires `funct3` = 000.
- SYSTEM with `funct3` = 000 requires `csr_addr` to be 0x000, 0x001 or 0x302; everything else — including 0x105 (`wfi`) — is illegal. `funct3` = 100 has no encoding.
- `csr_we` is conditional for the set/clear forms: `csrrs/csrrc/csrrsi/csrrci` write only when `inst[19:15]` is non-zero. This is why `rs1` is a port on a decoder that otherwise never looks at register numbers.

Defense points.

- *Why does an illegal instruction not trap?* The design decision (DESIGN.md §5.5) is that an illegal instruction

executes as a NOP: `illegal` forces every other output to zero — a “safety net” block at the end of the `always` block does this explicitly — so no architectural side effect can leak out of the decoder. It keeps `valid = 1` but carries the `illegal` flag, and both the retire pulse and the trace are qualified with `valid && !illegal`, so it is neither counted nor traced. Adding an illegal-instruction trap would have meant a second synchronous trap cause and a second trap source in EX; it was scoped out, and the behaviour is documented rather than left undefined.

- *Why are don't-cares driven to fixed values?* `alu_src_b` is computed by the uniform rule `~(R-type | branch)`, so `ecall/ebreak/mret/CSR` ops all read 1 even though they never use the ALU B operand. The reason is testability: the decoder is then a *total function* of the instruction word, and `tb_control.v` can compare every output bit for all 46 encodings plus 120 illegal ones rather than having to mask off don't-cares.
- *Why is decode a single flat **case** rather than a ROM?* 46 encodings of ~20 output bits is small enough that synthesis produces better logic from the case statement, and the case statement is readable against the truth table in the design document. The table in `DESIGN.md §3` is machine-generated from `tools/gen_control_table.py`, which is also the source of `tb_control.v`'s vectors — so the document and the test cannot drift apart.

3.6 alu_ctrl.v — second-level ALU decode

Purpose. Reduce `alu_class + funct3 + inst[30]` to the 4-bit ALU opcode. It exists as a separate module because the same `funct3` table is shared by R-type and I-type instructions, with one crucial difference.

Port	Dir	Width	Meaning
<code>alu_class</code>	in	2	From <code>control.v</code> : 0 ADD, 1 R-type, 2 I-type, 3 LUI
<code>funct3</code>	in	3	<code>inst[14:12]</code>
<code>funct7_b30</code>	in	1	<code>inst[30]</code> — the add/sub and srl/sra selector bit
<code>alu_op</code>	out	4	ALU opcode (see <code>alu.v</code>)

The critical difference. In class 1 (R-type), `inst[30]` selects SUB for `funct3 = 000` and SRA for `funct3 = 101`. In class 2 (I-type), `inst[30]` is consulted **only** for `funct3 = 101`; `funct3 = 000` is unconditionally ADD.

Defense points.

- *Why?* `addi x1, x0, -1` encodes as `0xfff00093`, whose bit 30 is 1 because the immediate is all ones. A decoder that consults `inst[30]` for every I-type operation turns that `addi` into a `sub`, computing `x0 - (-1) = 1`, so `x1` ends up 1 instead of -1. This is the single most common RV32I decode bug, it is on the project's non-negotiable correctness checklist (`CLAUDE.md`), and `tb_control.v` carries hand-computed goldens for exactly this case (`DESIGN.md §10.3`).
- *Why is **srai** an exception?* Because for shift-immediates the upper seven bits are *not* part of the immediate — the ISA defines the shift amount as only `imm[4:0]` and reuses `inst[31:25]` as a function field. So `inst[30]` genuinely is a decode bit there, and `control.v` has already rejected any value of `inst[31:25]` other than `0x00` or `0x20`.
- *Why does this module have no error state?* Because `control.v` has already validated `funct7`. Illegal combinations arrive as `alu_class = 0` (force-ADD) with `illegal` raised, so `alu_ctrl` can stay a pure table.

3.7 imm_gen.v — immediate generator

Purpose. Produce all six immediate forms from the raw instruction word.

Port	Dir	Width	Meaning
<code>inst</code>	in	32	Raw instruction word
<code>imm_sel</code>	in	3	Format select from <code>control.v</code>
<code>imm</code>	out	32	The selected immediate

Port	Dir	Width	Meaning
------	-----	-------	---------

The six cases, verbatim from the RTL:

imm_sel	Format	Expression
0	I	{{20{inst[31]}}, inst[31:20]}
1	S	{{20{inst[31]}}, inst[31:25], inst[11:7]}
2	B	{{19{inst[31]}}, inst[31], inst[7], inst[30:25], inst[11:8], 1'b0}
3	U	{inst[31:12], 12'b0}
4	J	{{11{inst[31]}}, inst[31], inst[19:12], inst[20], inst[30:21], 1'b0}
5	Z	{27'b0, inst[19:15]}
other	—	32'b0

Defense points.

- *Which immediates are sign-extended?* Five of six. **Z is the only zero-extended immediate** — CSR zimm is an unsigned 5-bit value, so sign-extending it would make csrrsi x0, mstatus, 0x18 set 27 unintended bits.
- *Where does the 1'b0 in B and J come from?* The ISA does not encode bit 0 of a branch or jump offset because instructions are 4-byte aligned (§1.4). Appending the zero here is how the byte offset is reconstructed.
- *Why is B's bit 11 taken from inst[7] and J's from inst[20]?* Because those are the bit positions the format left free after keeping every other immediate bit in the same physical wire as the neighbouring format (§1.4). They are also the two classic extraction bugs, and tb_imm_gen.v has hand-computed goldens for both.
- *Why do the shift-immediates produce a “wrong-looking” value?* slli/srli/srai reuse the I select, so srai rd, rs1, 31 yields 0x0000041f — the funct7 bits are still in there. It is harmless because alu.v uses only b[4:0] as the shift amount. This is worth knowing because it looks like a bug in a waveform.
- *How is it verified?* 1865 vectors: 1846 derived from the golden ISS's own decoder plus 19 hand-computed boundary goldens (−2048, +2047, ±4096, ±1 MiB, 0x800). The design document's own warning was “imm_gen is where the bugs live”, and it was the first module unit-tested (CLAUDE.md build order step 2).

3.8 regfile.v — 32 × 32 register file

Purpose. The architectural register file: two asynchronous read ports for ID, one synchronous write port for WB.

Port	Dir	Width	Meaning
clk	in	1	Clock
we	in	1	Write enable from WB
waddr	in	5	Destination register
wdata	in	32	Value to write
raddr1 / raddr2	in	5, 5	Read addresses (inst[19:15] / inst[24:20])
rdata1 / rdata2	out	32, 32	Read data, combinational

Logic — three behaviours in two assigns and one always:

```

assign rdata1 = (raddr1 == 5'd0)      ? 32'b0 : // x0 reads zero
               (we && waddr == raddr1) ? wdata : // WB->ID bypass
               regs[raddr1];
always @(posedge clk) if (we && waddr != 5'd0) regs[waddr] <= wdata;

```

Defense points.

- *Why the WB→ID internal bypass?* Without it, an instruction in ID reading a register that the instruction in WB is writing *this same cycle* would get the stale value — the write lands at the end of the cycle, the read happens during it. The bypass covers hazard distance 3. Without it the design would need either a third forwarding path or a third stall class (DESIGN.md §6.1). The alternative implementation is to write on the negative clock edge; the bypass was chosen because it keeps the design single-edge, which is a synthesis and static-timing requirement in practice.
- *Why waddr != 5'd0 on the write and raddr == 5'd0 on the read?* Belt and braces for x0. The read mux makes x0 read zero even if the array somehow held something; the write guard means the array never holds anything for entry 0. Note the bypass term is also implicitly safe because a write to x0 is suppressed — but the forwarding unit has its own rd != 0 guard as well (§3.10), because that is a different module with no visibility into this one.
- *Why asynchronous read?* Because ID must read, decode and reach the ID/EX register in a single cycle. A synchronous read would push the register value into EX and change the whole forwarding structure.
- *Why is it LUT RAM rather than block RAM?* Two asynchronous read ports are precisely what distributed (LUT) RAM provides and block RAM does not — a BRAM read is clocked. It is also tiny: $32 \times 32 = 1024$ bits, against a RAMB36E1's 36 Kbit. Putting it in a BRAM would waste an entire block RAM the memories need and would force a pipeline change. (* ram_style = "distributed" *) makes that explicit, because Vivado was observed to do exactly the wrong thing without it (§6.3).

3.9 id_ex.v — ID/EX pipeline register

Purpose. Carry the decoded control word, the register read values, the immediate and the bookkeeping fields from ID into EX. 212 payload bits (the PW localparam is the sum of every field width, used only to build the all-zero bubble constant).

Port group	Ports	Width	Meaning
Control	clk, rst, stall, flush	1 each	flush writes a bubble and outranks stall
Bookkeeping	valid_d/q, illegal_d/q, pc_d/q, inst_d/q	1,1,32,32	Retire/trace qualification; pc for link, branch base, mepc; inst for the trace
Reg addresses	rs1_addr_d/q, rs2_addr_d/q, rd_addr_d/q	5 each	Forwarding comparisons and write-back
Data	rs1_val_d/q, rs2_val_d/q, imm_d/q	32 each	Pre-forwarding values; imm also carries zimm
EX control	alu_op_d/q (4), alu_src_a_d/q, alu_src_b_d/q, funct3_d/q (3), branch_d/q, jalr_d/q, pred_taken_d/q	—	ALU op and operand selects; branch condition code; whether ID acted on a taken prediction
MEM control	mem_re_d/q, mem_we_d/q	1, 1	Passed through
WB control	reg_we_d/q, wb_sel_d/q	1, 2	Passed through
CSR/system	csr_en_d/q, csr_we_d/q, csr_imm_d/q, csr_addr_d/q (12), mret_d/q, ecall_d/q, ebreak_d/q	—	CSR RMW, trap entry, mret, halt

How `cpu_top` drives it. `stall` is tied to `halt` (freeze on `ebreak`), and `flush` is `~halt & (stall | flush_id)` — note that the *load-use stall* is what inserts the bubble here, because stalling ID/EX would hold the instruction in EX a second time rather than creating a gap. This is the “a stall in ID is a bubble in EX” mechanism.

Defense point. *Why does `pred_taken` travel but the predicted target does not?* The target is `ex_pc + ex_imm`, and both are already in this register, so EX can recompute it with an adder it already has. Carrying it would duplicate 32 flip-flops for nothing.

3.10 `forward_unit.v` — EX operand bypass selects

Purpose. Decide, for each of the three EX-stage operands, whether it comes from the ID/EX register, from EX/MEM, or from the WB write data. Purely combinational.

Parameter. `FORWARDING` — 0 forces every select to 2'b00 (the CPI-comparison build).

Port	Dir	Width	Meaning
<code>ex_rs1 / ex_rs2</code>	in	5, 5	Source registers of the instruction in EX
<code>mem_reg_we / mem_rd</code>	in	1, 5	Does the instruction in MEM write, and to where
<code>wb_reg_we / wb_rd</code>	in	1, 5	Does the instruction in WB write, and to where
<code>fwd_a</code>	out	2	Select for ALU operand A / branch <code>rs1</code> / CSR write source
<code>fwd_b</code>	out	2	Select for ALU operand B <i>and</i> branch <code>rs2</code> (taken before the <code>rs2/imm</code> mux)
<code>fwd_c</code>	out	2	Select for store data on its way to <code>dmem.wdata</code>

Select encoding. 2'b00 = ID/EX value, 2'b10 = EX/MEM res, 2'b01 = WB write data. (The encoding is not arbitrary: bit 1 means “the newer producer”).

Logic:

```
wire mem_writes = FWD_ON && mem_reg_we && (mem_rd != 5'd0);
wire wb_writes  = FWD_ON && wb_reg_we  && (wb_rd  != 5'd0);
assign fwd_a = (mem_writes && mem_rd == ex_rs1) ? 2'b10 :
               (wb_writes  && wb_rd  == ex_rs1) ? 2'b01 : 2'b00;
```

...and identically for `fwd_b` and `fwd_c` on `ex_rs2`.

Defense points.

- *Why three selects when `fwd_b` and `fwd_c` compute the same function?* They are kept apart so the store-data path is visible in the datapath and testable on its own (`asm/hazard/store_data_fwd.s`). Synthesis will share the logic if it wants to; the cost is zero and the clarity is real.
- *Why is `fwd_b` taken before the `rs2/immediate` mux?* Because a branch compares `rs2` even though the ALU is looking at the immediate for the same instruction. If the forward happened after the mux, a branch with a forwarded `rs2` would compare against an immediate.
- *Why does EX/MEM outrank MEM/WB?* See §5.2 — it is the newer-writer rule, and it is one of the two load-bearing rules in the module.
- *Why is `rd != 0` on the producer side enough — should the consumer be checked too?* If `mem_rd != 0` and `mem_rd == ex_rs1`, then `ex_rs1 != 0` follows. So the producer-side test implies the consumer-side test and a second comparison would be redundant.

- *Why does this unit not handle a load in EX/MEM?* Because `EX/MEM.res` is the ALU output, which for a load is the *effective address*, not the loaded data. The load-use interlock (§5.3) guarantees `2'b10` is never selected for a load; by the time a consumer reaches EX, the load is in WB and `2'b01` carries real data.

3.11 `alu.v` — the arithmetic/logic unit

Purpose. All integer computation. Purely combinational, 4-bit opcode.

Port	Dir	Width	Meaning
a	in	32	Operand A: forwarded <code>rs1</code> , or the PC for <code>auipc</code>
b	in	32	Operand B: forwarded <code>rs2</code> or the immediate — already muxed
alu_op	in	4	Operation select
y	out	32	Result

alu_op	Name	Result	Used by
0	ADD	<code>a + b</code>	ALU adds, all address computation, <code>auipc</code> , <code>link</code> , <code>jalr</code> target
1	SUB	<code>a - b</code>	<code>sub</code>
2	SLL	<code>a << b[4:0]</code>	<code>sll</code> , <code>slli</code>
3	SLT	<code>\$signed(a) < \$signed(b)</code>	<code>slt</code> , <code>slti</code>
4	SLTU	<code>a < b</code> (unsigned)	<code>sltu</code> , <code>sltiu</code>
5	XOR	<code>a ^ b</code>	<code>xor</code> , <code>xori</code>
6	SRL	<code>a >> b[4:0]</code>	<code>srl</code> , <code>srli</code>
7	SRA	<code>\$signed(a) >>> b[4:0]</code>	<code>sra</code> , <code>srai</code>
8	OR	<code>a b</code>	<code>or</code> , <code>ori</code>
9	AND	<code>a & b</code>	<code>and</code> , <code>andi</code>
10	PASSB	<code>b</code>	<code>lui</code>
other	—	<code>32'b0</code>	unreachable

Defense points.

- *Why is the shift amount `b[4:0]` rather than a separate port?* Because `b` is already the muxed operand: for a register shift it is `rs2` (so `rs2[4:0]` is the shift amount, as the ISA requires) and for an immediate shift it is the I-immediate (so `imm[4:0]`). One expression covers both cases correctly, which is why this design cannot exhibit the classic “register shifts use the wrong field” bug.
- *Why `$signed(a) >>> b[4:0]` for SRA?* The `>>>` operator is an arithmetic shift only when its left operand is signed; on an unsigned operand Verilog does a logical shift. Omitting `$signed` is a real bug and it is one of the mutations that was deliberately injected and confirmed caught by `tb_alu` (DESIGN.md §10.4).
- *Why is PASSB a separate opcode rather than `add a, 0`?* Because it makes `lui`’s operand-A path a don’t-care. With PASSB the decoder does not have to force `rs1 = x0` or zero the A operand, and the intent (“the result is the immediate”) is visible.
- *Why are branch comparisons not done here?* Keeping them in `branch_unit.v` keeps the branch decision off the ALU’s critical path. The ALU and the comparator then run in parallel in EX rather than in series — which matters, because the comparator is on the critical path as it is (§6.4).
- *How is it verified?* 5072 vectors across all 11 operations, signed/unsigned extremes and every shift amount 0–31 (DESIGN.md §10.3).

3.12 branch_unit.v — branch condition evaluation

Purpose. Evaluate the six conditional-branch tests on the *forwarded* register values.

Port	Dir	Width	Meaning
rs1	in	32	Forwarded operand A (<code>ex_a_val</code>)
rs2	in	32	Forwarded operand B (<code>ex_b_val</code> , taken before the immediate mux)
funct3	in	3	Branch condition code
taken	out	1	Condition result

funct3	Instruction	Test
000	beq	<code>rs1 == rs2</code>
001	bne	<code>rs1 != rs2</code>
100	blt	<code>\$signed(rs1) < \$signed(rs2)</code>
101	bge	<code>\$signed(rs1) >= \$signed(rs2)</code>
110	bltu	<code>rs1 < rs2</code> (unsigned)
111	bgeu	<code>rs1 >= rs2</code> (unsigned)
010, 011	—	0 (rejected as illegal by <code>control.v</code>)

Defense points.

- *Why does it take the forwarded values rather than the ID/EX values?* Because a branch is a consumer like any other instruction. `beq` immediately after an `addi` that computes one of its operands must compare the forwarded value, or the branch goes the wrong way. `cpu_top.v` wires `.rs1(ex_a_val)` and `.rs2(ex_b_val)` — the *outputs of the forwarding muxes*.
- *Why the signed/unsigned distinction?* `blt` on `-1` and `1` must be taken (`-1 < 1`); `bltu` on the same bit patterns must not (`0xFFFFFFFF > 1`). These comparisons are the same hardware with a different interpretation of the top bit, and `tb_branch_unit.v` runs 16072 vectors across the signed/unsigned boundary cases.
- *Where does **taken** go?* Into `br_redirect` in EX (mispredict detection) and into `bht.update_taken` (predictor training). It is the same signal for both, which is why training can never disagree with the architectural outcome.

3.13 csr.v — CSR file, trap entry, and mret

Purpose. Hold the machine-mode CSR subset, perform CSR read-modify-write in EX, implement trap entry and `mret`, and qualify the external interrupt.

Port	Dir	Width	Meaning
clk, rst	in	1, 1	Clock; reset zeroes every CSR
csr_en	in	1	A CSR instruction is in EX (enables the read side)
csr_we	in	1	Its write side actually happens
csr_addr	in	12	CSR number
csr_op	in	2	<code>funct3[1:0]</code> : 01 = RW, 10 = RS, 11 = RC
csr_wsrc	in	32	Write source: forwarded <code>rs1</code> , or <code>zimm</code>

Port	Dir	Width	Meaning
<code>csr_rdata</code>	out	32	Pre-write value; 0 for unimplemented CSRs or when <code>csr_en = 0</code>
<code>trap</code>	in	1	Take a trap this cycle (raised by <code>cpu_top</code>)
<code>trap_pc</code>	in	32	EX-stage PC of the faulting/interrupted instruction
<code>trap_cause</code>	in	32	11 for <code>ecall</code> , 0x8000000B for an external interrupt
<code>mret</code>	in	1	An <code>mret</code> is in EX
<code>mtvec_o</code>	out	32	Trap vector, to the PC-select mux
<code>mepc_o</code>	out	32	Return address, to the PC-select mux
<code>irq</code>	in	1	External interrupt request — a level , not a pulse
<code>irq_pending</code>	out	1	<code>irq & mstatus.MIE & mie.MEIE</code>

Implemented state:

CSR	Address	Bits implemented
<code>mstatus</code>	0x300	MIE = bit 3, MPIE = bit 7; all others read 0
<code>mie</code>	0x304	MEIE = bit 11 only
<code>mtvec</code>	0x305	[31:2]; [1:0] always read 00 (direct mode)
<code>mepc</code>	0x341	[31:2]; [1:0] always read 00
<code>mcause</code>	0x342	all 32 bits, hardware- and software-writable

Write-side priority: `trap` > `mret` > CSR instruction. They are mutually exclusive in practice (one instruction occupies EX, and `cpu_top` suppresses `csr_we` when a trap squashes it), but the explicit priority makes an interrupt-squashes-a-CSR-instruction case safe by construction.

Trap entry writes `mepc` ← {`trap_pc`[31:2], 2'b00}, `mcause` ← `trap_cause`, `MPIE` ← `MIE`, `MIE` ← 0. **`mret`** writes `MIE` ← `MPIE`, `MPIE` ← 1.

Defense points.

- *Why is there no CSR hazard interlock?* Because the read-modify-write is entirely within one EX cycle: `csr_rdata` is the pre-write architectural value and the new value is committed on the same clock edge. An instruction one slot behind reaches EX a cycle later and reads the updated register. This is why `csrw mepc, t0` immediately followed by `mret` works, which the demonstration ISR relies on.
- *What does need forwarding, then?* The **register** side: `csr_wsrc` is `ex_csr_imm ? ex_imm : ex_a_val` — the forwarded `rs1`, not the raw ID/EX value. And the destination register gets the old CSR value via the normal `wb_sel` = CSR path, which is why the result mux is in EX (§2.2).
- *Why does `irq` get qualified inside this module?* Because MIE and MEIE live here. Doing it here means a program that never enables interrupts is provably unaffected by the `irq` line: no trap, no `mcause` write, not one extra cycle — which is exactly what running `fib/bsort/bloop` with `irq` held high demonstrates (DESIGN.md §11.4).
- *Why are `mtvec`[1:0] and `mepc`[1:0] forced to zero?* `mtvec`[1:0] is the *mode* field in the privileged spec: 00 = direct, 01 = vectored. Hard-wiring 00 is the architecturally correct way to say “this implementation supports direct mode only”. For `mepc`, the low bits must be zero because every instruction is 4-byte aligned, so storing them would store known constants. Both also save flip-flops.

3.14 ex_mem.v — EX/MEM pipeline register

144 payload bits. Fields and rationale are in §2.5; the port list is the `_d/_q` pair for each field plus `clk`, `rst`, `stall`, `flush`.

Port	Dir	Width	Meaning
<code>valid_d/q</code> , <code>illegal_d/q</code>	in/out	1, 1	Retire/trace qualification
<code>pc_d/q</code> , <code>inst_d/q</code>	in/out	32, 32	Commit trace
<code>rd_addr_d/q</code> , <code>reg_we_d/q</code>	in/out	5, 1	Write-back target; EX/MEM forwarding match
<code>wb_sel_d/q</code>	in/out	2	Load data vs <code>res</code> in WB
<code>res_d/q</code>	in/out	32	ALU result / <code>PC+4</code> / old CSR value / effective address
<code>store_data_d/q</code>	in/out	32	Forwarded <code>rs2</code> for stores
<code>funct3_d/q</code>	in/out	3	Load/store width and signedness
<code>mem_re_d/q</code> , <code>mem_we_d/q</code>	in/out	1, 1	Drive <code>dmem.v</code>
<code>ebreak_d/q</code>	in/out	1	Halt on retirement

How `cpu_top` drives it. `stall` is `halt`; `flush` is `flush_ex = trap_taken & ~halt` — *this* is where a trapping instruction is squashed so that it never retires. Note also that `reg_we_d`, `mem_re_d` and `mem_we_d` are all ANDed with `ex_valid` on the way in, so a bubble in EX can never produce a memory access or a register write even if a field were somehow non-zero.

3.15 dmem.v — data memory with byte lanes

Purpose. 4 KB read/write data memory. Byte/halfword lane handling and load extension live inside the module rather than in the datapath.

Parameter. `INIT` — `$readmemh` image, or "" for zero-filled.

Port	Dir	Width	Meaning
<code>clk</code>	in	1	Clock
<code>we</code>	in	1	Store this cycle (MEM stage)
<code>re</code>	in	1	Load this cycle (MEM stage)
<code>addr</code>	in	32	Effective byte address (EX/MEM. <code>res</code>); word index <code>addr[11:2]</code> , lane <code>addr[1:0]</code>
<code>funct3</code>	in	3	Width/signedness: 000 b, 001 h, 010 w, 100 bu, 101 hu
<code>wdata</code>	in	32	Store data — the forwarded rs2
<code>rdata</code>	out	32	Load result, lane-selected and extended; valid the cycle after <code>re</code>
<code>wmask_data</code>	out	32	Store data masked to width, unshifted — commit trace only

Logic in three parts.

1. *Write*: a case (`funct3[1:0]`) selects `sb` (one of four byte lanes by `addr[1:0]`), `sh` (upper or lower half by `addr[1]`) or `sw` (the whole word).
2. *Read*: on the clock edge, `rword` $\leftarrow$ `mem[addr[11:2]]`, `raddr_1` $\leftarrow$ `addr[1:0]`, `rfunct3` $\leftarrow$ `funct3`. The lane select and sign/zero extension are then combinational on those registered values, so `rdata` is valid during WB.
3. *wmask_data*: a pure observation output, masked to width so the trace line for `sb x6`, `0(x5)` prints `mem[...]=000000ff` rather than the full register.

Defense points.

- *Why register the raw word and extend combinatorially, rather than registering the extended value?* Because the extension depends on `funct3` and `addr[1:0]`, which would otherwise also have to cross into WB — and because it lets the load result be consumed in WB without a second pipeline stage. The cost is that the extension logic is in the same cycle as the 2.454 ns BRAM read, which is steps 1–2 of the critical path (§6.4). This is a real, measured trade-off and the first named fix is to undo it.
- *Little-endian?* Yes: byte 0 occupies bits [7:0]. `lb/lh` sign-extend, `lbu/lhu` zero-extend. This is the RISC-V standard and it is what the reference model implements.
- *What happens on a misaligned access?* It is architecturally **don't-care** in this implementation: no trap is raised, the word index is `addr[11:2]` and the lane comes from `addr[1:0]`. A misaligned `lw` therefore reads the containing word. The reference simulator models the identical behaviour, so the two still agree bit-for-bit. Real RV32I would raise a misaligned-access exception or emulate it; this is a documented simplification (DESIGN.md §7), not an accident.
- *Why is `wdata` described as “already forwarded”?* Because store-data forwarding happens in EX via `fwd_c`, before EX/MEM.`store_data`. By the time the value arrives here it is correct by construction, so the memory itself needs no hazard awareness.
- *Why `ram_style = "block"`?* Left alone, Vivado put this array in 512 RAMS64E distributed-RAM cells (§6.3). Forcing block RAM is what makes the design's resource figure honest, and it is also what put a 2.454 ns clock-to-out at the head of the critical path — those two facts are the same decision.

3.16 mem_wb.v — MEM/WB pipeline register

140 payload bits.

Port	Dir	Width	Meaning
<code>valid_d/q</code> , <code>illegal_d/q</code>	in/out	1, 1	Qualify retire and the trace
<code>pc_d/q</code> , <code>inst_d/q</code>	in/out	32, 32	Commit trace
<code>rd_addr_d/q</code> , <code>reg_we_d/q</code>	in/out	5, 1	Register-file write port; MEM/WB forwarding match
<code>wb_sel_d/q</code>	in/out	2	Write-back mux select
<code>res_d/q</code>	in/out	32	Non-memory result; also the trace's store address
<code>mem_we_d/q</code>	in/out	1	A store retired (trace)
<code>mem_val_d/q</code>	in/out	32	Store data masked to width (trace)
<code>ebreak_d/q</code>	in/out	1	Sticky halt

`flush` is tied to `1'b0` in `cpu_top`: nothing ever flushes MEM/WB, because by the time an instruction is in WB it is older than any redirect source and is architecturally committed. `stall` is `halt`.

Defense point. *Why is the load data not a field here?* `dmem.v` already registered the memory word on the MEM edge and presents the extended value combinatorially during WB, so a second register would duplicate 32 flip-flops and add nothing.

3.17 hazard_unit.v — interlock and flush generation

Purpose. Produce the three pipeline-control signals. Purely combinational.

Parameter. FORWARDING — selects the load-use rule (1) or the stall-on-any-RAW rule (0).

Port	Dir	Width	Meaning
id_valid	in	1	IF/ID holds a real instruction
id_rs1, id_rs2	in	5, 5	Source register numbers of the instruction in ID
id_uses_rs1, id_uses_rs2	in	1, 1	Decoder's answer to “is that field really a source?”
ex_valid, ex_mem_re, ex_reg_we, ex_rd	in	1,1,1,5	The instruction in EX: is it valid, is it a load, does it write, and where
mem_valid, mem_reg_we, mem_rd	in	1,1,5	The instruction in MEM (only used when FORWARDING = 0)
redirect_ex	in	1	Taken branch / jalr / trap / mret
redirect_id	in	1	jal or a predicted-taken branch
ebreak_pending	in	1	An ebreak is in EX, MEM or WB
stall	out	1	Hold PC + IF/ID, bubble ID/EX
flush_if	out	1	Clear IF/ID to a bubble
flush_id	out	1	Clear ID/EX to a bubble

Logic:

```

ex_match = (id_uses_rs1 && id_rs1 == ex_rd) || (id_uses_rs2 && id_rs2 == ex_rd);
mem_match = (id_uses_rs1 && id_rs1 == mem_rd) || (id_uses_rs2 && id_rs2 == mem_rd);

load_use = ex_valid && ex_mem_re && ex_rd != 0 && ex_match;           // FWD = 1
raw_any  = (ex_valid && ex_reg_we && ex_rd != 0 && ex_match) ||      // FWD = 0
            (mem_valid && mem_reg_we && mem_rd != 0 && mem_match);

stall     = id_valid && (FWD_ON ? load_use : raw_any)
            && !redirect_ex && !ebreak_pending;
flush_if  = redirect_ex || redirect_id || ebreak_pending;
flush_id  = redirect_ex || ebreak_pending;

```

Defense points.

- *Why does **flush_id** not include **redirect_id**?* Because a **jal** in ID is itself the redirecting instruction and must continue into EX to compute and write its link value. Only the instruction behind it (in IF) is on the wrong path. That is the whole 1-bubble-versus-2-bubble distinction.
- *Why are **id_uses_rs1**/**id_uses_rs2** separate inputs rather than derived from the instruction bits?* Because **lui**, **auipc**, **jal**, the **csrr*i*** forms and **ecall** have *something* in the **rs1/rs2** bit positions — part of an immediate, or nothing at all — and treating those bits as register numbers would raise stalls for dependencies that do not exist. That costs cycles, not correctness, but the counters exist to measure CPI, so they must not count phantom hazards. This was mutation-tested: removing the qualifiers is still functionally correct everywhere but costs cycles (**bloop** at FORWARDING=0: 5313 → 5515), DESIGN.md §10.4.
- *Why is **redirect_id** an input if it is not used in **stall**?* It feeds **flush_if**. Its deliberate *absence* from **stall** is the fix for a combinational loop — see §5.4, which is the most interesting bug in this design.
- *Why does **ebreak_pending** both stall-suppress and flush?* An **ebreak** must let nothing behind it commit, and holding IF/ID and ID/EX empty for the three cycles it takes to drain is how that is done. Suppressing **stall** at the same time prevents a stall from fighting the drain.

3.18 perf_counters.v — performance counters

Purpose. Six free-running 32-bit counters, the measurement instrument for §6 and §7’s numbers. It is deliberately “dumb”: `cpu_top` gates the pulses.

Port	Dir	Width	Meaning
<code>clk, rst</code>	in	1, 1	Clock; reset clears all six
<code>retire</code>	in	1	A valid, non-illegal instruction left WB
<code>lu_stall</code>	in	1	An interlock cycle
<code>flush</code>	in	1	A control-flow redirect event
<code>bht_pred</code>	in	1	A conditional branch resolved in EX
<code>bht_miss</code>	in	1	...and it was mispredicted
<code>cycles</code>	out	32	Every cycle after reset
<code>insns</code>	out	32	Retired instructions
<code>lu_stalls</code>	out	32	Interlock cycles
<code>flushes</code>	out	32	Redirect events
<code>bht_preds, bht_misses</code>	out	32, 32	Branch prediction totals

Counting conventions — these are the ones to know, because they define the reported numbers:

- `insns` counts *valid* && *!illegal* leaving WB. `ebreak` is included; an `ecall` squashed by its own trap is not (it never reaches WB).
- `lu_stalls` counts *cycles*, not events, and its meaning changes with the build: at `FORWARDING = 1` it is load-use cycles, at `FORWARDING = 0` it is every RAW stall cycle. That difference is the CPI experiment.
- `flushes` counts one per redirect **event**, not per killed slot. A taken branch that kills two instructions is one flush.
- The `ebreak` drain shadow is *not* counted as a flush — it is not a mispredicted control transfer.
- `bht_preds` counts every conditional branch resolved in EX at **either** `BHT_ENABLE` setting, so with the predictor off every taken branch registers as a “miss”. That is exactly the static-not-taken baseline the comparison needs.
- Every pulse is gated with `~halt`, so counting stops exactly at the `ebreak`.

CPI is `cycles / insns`, printed by the testbench as `CPI_x1000` to stay in integer arithmetic.

3.19 cpu_top.v — the top level

Purpose. Instantiate and wire the other 18 modules, and implement the logic that belongs to no single one of them: the PC-select mux, the forwarding muxes, the trap conditions, the result mux, the write-back mux, the halt flag, the commit-trace port.

Parameters:

Parameter	Default	Effect
<code>FORWARDING</code>	1	0 disables all bypasses; the hazard unit stalls on any RAW instead
<code>BHT_ENABLE</code>	1	0 forces every prediction to not-taken and writes no counter
<code>IMEM_INIT</code>	<code>"prog.hex"</code>	<code>\$readmemh</code> image for instruction memory
<code>DMEM_INIT</code>	<code>""</code>	<code>\$readmemh</code> image for data memory; empty = zero-filled

Ports (363 in total, of which 361 bits are observation-only):

Port	Dir	Width	Meaning
clk	in	1	Clock
rst	in	1	Synchronous, active-high
irq	in	1	External interrupt request, level-sensitive
done	out	1	Sticky: an ebreak retired; the machine is frozen
trace_valid	out	1	An instruction retired this cycle
trace_pc	out	32	Its PC
trace_insn	out	32	Its instruction word
trace_rd_we	out	1	It wrote a register (and rd != 0)
trace_rd	out	5	Which register
trace_rd_val	out	32	The value written
trace_mem_we	out	1	It was a store
trace_mem_addr	out	32	Effective byte address (MEM/WB.res)
trace_mem_val	out	32	Store data masked to width
perf_cycles ... perf_bht_miss	out	32 each	The six counters of §3.18

The logic that lives here and nowhere else:

Signal	Expression	Why
fetch_en	<code>~halt & (redirect \ ~stall)</code>	Drives PC, IMEM and IF/ID together; a redirect beats a stall
id_inst	<code>if_id_valid ? if_inst : NOP_INST</code>	A flushed slot decodes as a NOP regardless of what IMEM's output register holds
id_uses_rs1	R/I/load/store/branch/jalr, or a register-form CSR op	Feeds the hazard unit (§3.17)
id_uses_rs2	R-type, store, branch	Only these three actually read rs2
id_pred_taken	<code>if_id_valid & id_branch & if_id_pred_state[1]</code>	The BHT prediction, applied only to a real conditional branch
jal_taken	<code>if_id_valid & id_jal & ~stall & ~ebreak_pending</code>	ID redirect, gated so it cannot fire on a stalled cycle (§5.4)
bht_taken	<code>id_pred_taken & ~stall & ~ebreak_pending</code>	Same gating, same reason
ex_a_val / ex_b_val / ex_store_data alu_a / alu_b	3-way muxes on <code>fwd_a/fwd_b/fwd_c</code> <code>ex_alu_src_a ? ex_pc : ex_a_val / ex_alu_src_b ? ex_imm : ex_b_val</code>	The bypass network itself Operand selection after forwarding
irq_taken	<code>ex_valid & csr_irq_pending & ~ebreak_pending</code>	An interrupt attaches only to a real instruction, so mepc is always a genuine PC
trap_taken	<code>irq_taken \ (ex_valid & ex_ecall)</code>	The two trap sources; the interrupt outranks a co-located ecall
ex_branch_valid	<code>ex_valid & ex_branch & ~trap_taken</code>	A branch squashed by a trap must not train the BHT or count

Signal	Expression	Why
br_redirect	ex_branch_valid & (ex_pred_taken != branch_cond)	Redirect only on a mispredict — with BHT_ENABLE = 0 this degenerates to “redirect whenever taken”
ex_jalr_target	alu_y & ~32'h1	The ISA requires clearing bit 0 of a jalr target
ex_correct_target	branch_cond ? ex_pc + ex_imm : ex_pc + 4	Where a mispredicted branch has to go — either direction
ebreak_pending	ebreak in EX, MEM or WB	A continuous drain shadow, not a one-cycle signal
wb_data	(wbs_wb_sel == WB_MEM) ? dmem_rdata : wbs_res	The final write-back mux — only two ways, because the other three were resolved in EX
wb_reg_we	wbs_valid & wbs_reg_we & (rd != 0) & ~halt	x0 suppression, second line of defense
done_r	sticky on wbs_valid & wbs_ebreak	Halt

Defense point. *Why is so much logic at the top level rather than in modules?* Every expression in that table is a *relationship between modules* — a mux whose inputs come from three different stages, or a condition combining decoder output with pipeline state. Pushing them into a module would mean routing the same signals in and back out again. The rule followed is: a module owns a thing (a memory, a table, a decoder); `cpu_top` owns the wiring and the arbitration between things.

4. Why N bits — every width decision

4.1 Why a 32-bit datapath

Why not 64? RV64I would double the width of the register file, the ALU, every pipeline register’s data fields, both memories and the entire forwarding network — the 32-bit `res`, `store_data`, `rs1_val`, `rs2_val` and `imm` fields alone are 160 bits per instruction in flight — for a machine whose largest test program addresses 4 KB of data and computes `fib(30) = 832040`. Nothing in the workload needs more than 32 bits of address or data. The cost would be roughly double the flip-flops and a longer carry chain in the ALU and in the branch comparator, which sits on the critical path (§6.4) — so a 64-bit version of this design would be both bigger and slower with no functional gain.

Why not 16? Because 32 bits is what the ISA defines and what the encoding assumes: `lui` places a 20-bit immediate at bits [31:12], which only makes sense in a 32-bit register; the shift amount is 5 bits because $2^5 = 32$; `auipc`’s PC-relative addressing assumes a 32-bit PC. A 16-bit datapath would not be RV32I, it would be a different ISA. It would also make every 32-bit constant a multi-instruction sequence and every memory word a two-cycle access.

Why 32 is the right size for this machine. It addresses 4 GiB (far more than the 4 KB actually implemented, so the address space is not a constraint), it holds the full range of a signed integer benchmark, and it matches the natural width of the FPGA’s resources — the `RAMB36E1` is 36 Kbit and maps naturally to 1024×32 , and the `CARRY4` primitive chains four bits at a time so a 32-bit adder is exactly eight `CARRY4`s.

4.2 Why 32 registers, and why x0 is hardwired to zero

Why 32? It is the number the ISA specifies, and the reason it is 32 rather than 16 or 64 is an encoding argument: the register field is 5 bits ($2^5 = 32$), and R-type instructions need three of them — `rs1`, `rs2`, `rd` = 15 bits — plus a 7-bit opcode, a 3-bit `funct3` and a 7-bit `funct7`, which is exactly 32 bits. Any more registers and R-type would not fit in a word. Any fewer and the compiler would spill to memory more often.

Cost in this design: `regfile.v` holds $32 \times 32 = 1024$ bits, realised as 44 LUTs of distributed RAM (§6.2). That is under 3 % of the design's LUTs, so the register file is essentially free — the expensive thing is not the storage but the two asynchronous read ports, which is why it must be distributed RAM (§3.8).

Why x0 is hardwired to zero — four separate payoffs:

1. *A source of zero costs no instruction.* `add x5, x0, x0` zeroes a register; `beq x5, x0, label` compares against zero; `jalr x0, 0(x1)` is `ret`.
2. *A discard destination.* Writing to `x0` throws the result away, which is how `csrrw x0, mtvec, t0` performs a CSR write with no register side effect, and how `ebreak`-adjacent idioms work.
3. *Pseudo-instructions collapse onto it.* `nop` is `addi x0, x0, 0` — which is precisely the `NOP_INST = 32'h00000013` constant `cpu_top.v` substitutes for a flushed slot. `mv rd, rs` is `addi rd, rs, 0`. `j label` is `jal x0, label`.
4. *It simplifies hazard logic.* An instruction writing `x0` produces no dependency, so every hazard comparison can be pruned with a single `rd != 0` term.

Where x0 is enforced, and why in more than one place. `regfile.v` suppresses both the write (if (we && waddr != 5'd0)) and the read (`raddr == 0 ? 32'b0`). `forward_unit.v` independently requires `rd != 0` before forwarding. `hazard_unit.v` independently requires `rd != 0` before stalling. `cpu_top.v` requires it again for `wb_reg_we` and for `trace_rd_we`. These are not redundant checks of the same thing: they are four different modules, each of which would be wrong on its own without it. The forwarding one is the subtle case — if the forwarding unit forwarded from an instruction whose `rd` is `x0`, then `x0` would read non-zero for exactly one instruction even though the register file never stored anything. `asm/hazard/x0_hazard.s` writes `x0` and immediately reads it, and that test exists specifically because this mutation was injected and had to be caught (DESIGN.md §10.4).

4.3 The immediate formats, bit by bit

Format	Encoded bits	Reconstructed value	Range
I	<code>inst[31:20]</code> (12)	sign-extend to 32	−2048 ... +2047
S	<code>inst[31:25]</code> , <code>inst[11:7]</code> (12)	sign-extend to 32	−2048 ... +2047
B	<code>inst[31]</code> , <code>inst[7]</code> , <code>inst[30:25]</code> , <code>inst[11:8]</code> (12) + implicit 0	sign-extend, LSB = 0	−4096 ... +4094, even
U	<code>inst[31:12]</code> (20)	placed at [31:12], low 12 zero	0 ... 0xFFFFF000, step 4096
J	<code>inst[31]</code> , <code>inst[19:12]</code> , <code>inst[20]</code> , <code>inst[30:21]</code> (20) + implicit 0	sign-extend, LSB = 0	−1 MiB ... +1 MiB−2, even
Z	<code>inst[19:15]</code> (5)	zero-extend	0 ... 31

Why B-type's bit 0 is always zero. Every instruction is 4 bytes and 4-byte aligned, so any branch target is even — in fact a multiple of 4. Encoding bit 0 would spend an encoding bit on a value that is always zero. Dropping it and appending `1'b0` in the generator doubles the reach of the same 12 encoded bits from ± 2 KiB to ± 4 KiB. The generator's B case ends with `..., inst[11:8], 1'b0` — that trailing constant is the whole mechanism. (Note the range is ± 4 KiB, *not* ± 4 KiB in steps of 4: bit 1 *is* encoded, so a branch offset of 2 is representable even though no legal instruction sits there. The ISA chose to keep the field shape uniform with S rather than squeeze out a second bit.)

Why U-type places the immediate at [31:12] with no sign extension. The purpose of `lui` is to build the *upper* 20 bits of a 32-bit constant, so the immediate is the top of the word by definition and there is nothing above it to extend into. `inst[31]` still ends up as the result's sign bit, just by position rather than by replication. Combined with a sign-extended `addi`, the pair can build any 32-bit constant — with the carry correction described in §1.4, which `tools/asm.py` implements and `tools/test_tools.py` checks at the boundaries.

Why J-type shuffles its bits. Look at the encoded field order: `imm[20]` at `inst[31]`, `imm[10:1]` at `inst[30:21]`, `imm[11]` at `inst[20]`, `imm[19:12]` at `inst[19:12]`. Two things are being satisfied simultaneously: (a) the sign

bit must be `inst[31]`, like every other format, so `imm[20]` — the sign of a 21-bit offset — goes there; and (b) `imm[19:12]` stays in `inst[19:12]`, the same physical wires U-type uses for the corresponding part of its immediate. Everything else is packed into what is left. The result is that `imm_gen.v`'s J case and U case share wiring rather than needing independent routing. The apparent disorder is the *consequence* of two ordering constraints, not an arbitrary scramble — that is the answer if asked “why is it so messy?”.

4.4 Why 5-bit shift amounts

A 32-bit register can be shifted by 0 ... 31 positions, and 32 values need exactly 5 bits. A 6th bit would encode shift amounts of 32–63, which RV32I defines as *not* encodable — `slli` with `inst[25] = 1` is an illegal instruction, and `control.v` enforces that by requiring `inst[31:25] = 0x00` for `slli/srli` and `0x20` for `srai`.

The consequence in the datapath is that `alu.v` uses `b[4:0]` and ignores `b[31:5]` entirely. That single expression correctly covers both cases the ISA defines:

- **Register shifts** (`sll`, `srl`, `sra`) use `rs2[4:0]`, discarding the upper 27 bits. So `sll x5, x6, x7` with `x7 = 33` shifts by 1, not by 33 — this is defined ISA behaviour, not an accident, and `tb_alu` checks it.
- **Immediate shifts** (`slli`, `srli`, `srai`) use `imm[4:0]`, which is why it does not matter that `imm_gen` hands over the full sign-extended `inst[31:20]` including the `funct7` bits (§3.7).

Using `b[4:0]` for both is what makes the classic “register shift uses the wrong field” bug structurally impossible here: there is only one field.

4.5 Word-aligned PC, byte-addressed memory

The PC is a byte address that advances by 4. `pc_next = pc_q + 32'd4` in `cpu_top.v`. It is not an instruction index, because RISC-V memory is byte-addressed and PC-relative addressing (`auipc`, branches, `jal`) has to produce byte addresses that agree with data addresses.

But the low two bits are always zero, which is why:

- `imem.v` indexes with `addr[11:2]` and simply ignores `addr[1:0]`;
- `mepc` and `mtvec` store only `[31:2]` and read 00 in the low two bits (§3.13);
- the B and J immediates do not encode bit 0 (§4.3);
- `jalr` clears bit 0 explicitly (`ex_jalr_target = alu_y & ~32'h1`) because its target comes from a register that could hold anything — this is the ISA-mandated `& ~1`.

Data memory is byte-addressed with lanes inside `dmem.v`. The word index is `addr[11:2]` and the lane comes from `addr[1:0]`: `sb` picks one of four byte lanes, `sh` picks the upper or lower half, `sw` writes the whole word. Loads reverse the process, with sign extension for `lb/lh` and zero extension for `lbu/lhu`. This is what makes byte-level data manipulation possible on a word-organised memory, and it is the reason `funct3` has to travel all the way to MEM.

4.6 Why 4 KB IMEM and 4 KB DMEM

Consideration	Number
IMEM capacity	1024 instructions
Largest test program, static	<code>bpred.s</code> = 63 instructions = 252 bytes (6 % of IMEM)
Others	<code>fib</code> 176 B, <code>bsort</code> 144 B, <code>bloop</code> 100 B, <code>irq_demo</code> 108 B
DMEM capacity	1024 words; <code>sp</code> initialised to 0x00001000 (top, growing down)
Largest data use	<code>bsort</code> 's 16-word array plus stack frames

Why not smaller? 1 KB would still fit every program, but 4 KB costs nothing: a single `RAMB36E1` is 36 Kbit = 4.5 KB, so 4 KB is the largest power-of-two that fits in *one* block RAM per memory. Using 1 KB would occupy the same primitive and simply waste three quarters of it.

Why not larger? 8 KB would need two RAMB36E1 per memory, doubling the BRAM count from 2 to 4 for storage no program uses. The xc7a35t has 50 block RAMs so there is room, but reporting 4 BRAMs when 2 suffice would misrepresent the design’s cost.

The address-space consequence: with 4 KB, only `addr[11:2]` is decoded and the space wraps — address `0x1000` is the same word as `0x0`. That is documented as deliberate (DESIGN.md §7). It is also why `sp = 0x00001000` works as “top of memory”: the first `addi sp, sp, -4` brings it to `0x0FFC`, the top word.

4.7 BHT sizing: why 64 entries, 2 bits, and `pc[7:2]`

Why index with `pc[7:2]`? Bits `[1:0]` are always zero (§4.5), so they carry no information and indexing with them would leave three quarters of the table unused. The index therefore starts at bit 2. Six bits of index (`[7:2]`) select one of 64 entries and cover a **256-byte window** of instruction space.

Why 64 entries? It is the smallest table that gives every branch in a realistic test program its own counter. All of this project’s programs are under 256 bytes of static code — the largest, `bpred.s`, is 252 bytes — so for the measured workloads **no two branches ever share a counter**. A 32-entry table (`pc[6:2]`) would halve the window to 128 bytes and start aliasing inside `bpred`, `fib` and the larger per-instruction programs; a 256-entry table would cost 4× the flip-flops (512 instead of 128) to cover address space no test program reaches.

The aliasing trade-off, stated honestly. The table is **untagged**: there is no check that the entry actually belongs to the branch being looked up. Two branches 256 bytes apart share a counter and train against each other. This degrades *accuracy* and never *correctness* (§7.1), and it is the standard cost of a cheap predictor — a tagged table would need a comparator and a tag field per entry, roughly tripling the storage. For the programs measured here the effect is zero, because they are all smaller than the window; the mechanism is real but would only appear on a program larger than 256 bytes.

Why 2-bit counters rather than 1-bit? A 1-bit predictor simply remembers the last outcome. On a loop that runs n times, it mispredicts **twice** per execution of the loop: once on the first iteration (it remembers the previous loop’s exit) and once on the exit. A 2-bit saturating counter needs two consecutive contrary outcomes to change its prediction, so the single not-taken exit does not flip it — one mispredict per loop instead of two. For a loop nested inside another loop, that halving applies on every pass of the outer loop. This is the textbook result and it is why 2-bit is the standard minimum.

Why not 3-bit (or more)? Diminishing returns: a 3-bit counter needs three contrary outcomes to switch, which adds hysteresis that helps on very stable branches but *hurts* on branches that genuinely change behaviour, and it costs 50 % more storage (192 bits instead of 128). The measured accuracies here (95.5 % on `bpred`, 93.4 % on `fib`) are already close to the ceiling of what direction prediction without history can do; the remaining misses on `bloop` come from an alternating pattern that *no* saturating counter of any width can learn (§7.1), so more bits would not touch them.

Cost, measured: $64 \times 2 = 128$ flip-flops of counter state, and the full/base comparison gives +130 FF and +216 LUT for the predictor as a whole (`utilization.txt` vs `utilization_base.txt`) — the extra 2 flip-flops are the prediction state carried in IF/ID, and the LUTs are the saturating-update logic, the ID-stage redirect path and the EX-stage mispredict comparison.

4.8 CSR bit fields — why those bits

Field	Position	Why there
<code>mstatus.MIE</code>	bit 3	Fixed by the RISC-V privileged spec: the machine-mode interrupt enable
<code>mstatus.MPIE</code>	bit 7	Fixed by the spec: saves MIE across a trap
<code>mie.MEIE</code>	bit 11	Fixed by the spec: machine external interrupt enable (bit 3 would be software, bit 7 timer)
<code>mtvec[1:0]</code>	mode	00 = direct, 01 = vectored. Hardwired 00

Field	Position	Why there
<code>mtvec[31:2]</code>	BASE	Trap vector address, necessarily 4-byte aligned
<code>mepc[1:0]</code>	—	Always 00 — instructions are 4-byte aligned
<code>mcause</code> bit 31	interrupt flag	1 = interrupt, 0 = exception. Hence 0x8000000B
<code>mcause[30:0]</code>	cause code	11 = machine external interrupt; also 11 = environment call from M-mode

These positions are **not design choices** — they come from the RISC-V privileged specification, and the correct answer to “why bit 3?” is “because the specification says so, and deviating would mean the CSR names lie about what they are.” What *is* a design choice is which bits are implemented at all: everything else in `mstatus` reads zero and ignores writes, which the specification explicitly permits (a WARL field may be read-only-zero).

The 0x8000000B value is worth being able to take apart on the spot: bit 31 set means “this is an interrupt, not an exception”; the low bits 0xB = 11 is the machine external interrupt code. The `ecall` cause is plain 11 with bit 31 *clear* — the same number 11, meaning “environment call from M-mode”. Two different events sharing the code 11 with different interpretations of bit 31 is exactly how the specification defines it, and it is a good question to be ready for.

Why direct mode rather than vectored? In vectored mode the hardware computes `mtvec.BASE + 4 × cause` so different interrupt sources land on different handler entries. With exactly one interrupt source (the external `irq` line) plus `ecall`, a vector table would have one used entry and a lot of padding, and it would add a multiply-by-4-and-add to the trap path in EX — on a design whose critical path already ends in the PC mux. Direct mode is one input to that mux instead.

5. The hazard scheme

5.1 Why forward from EX/MEM and MEM/WB, and only those

An instruction reads its registers in ID, but the three instructions ahead of it have not written theirs back yet. Counting from the consumer:

Distance	Producer is in...	Handled by
1	EX (result at the EX/MEM boundary)	EX/MEM bypass, select 2'b10
2	MEM (result at the MEM/WB boundary)	MEM/WB bypass, select 2'b01
3	WB (writing the register file this cycle)	the register file's WB→ID internal bypass
4 or more	already retired	ordinary register read

So there are exactly **two** forwarding paths because there are exactly two pipeline stages between “result computed” and “result architecturally visible”. Distance 3 is covered inside `regfile.v` rather than by a third mux input — if it were not, the design would need either a third forwarding path or a third stall class (DESIGN.md §6.1).

Three *operands* need forwarding because three different consumers read a register in EX: the ALU's A input, the ALU's B input (which is also the branch comparator's `rs2`), and the store-data path. Hence `fwd_a`, `fwd_b`, `fwd_c`.

5.2 Why EX/MEM has priority over MEM/WB

This is the newer-writer rule, and it is the single most important line in `forward_unit.v`.

Consider:

```
addi x5, x0, 100    # writes 100
addi x5, x5, 1      # writes 101
add  x6, x5, x0      # must read 101
```

When the third instruction is in EX, the second is in MEM (EX/MEM holds 101) and the first is in WB (MEM/WB holds 100). **Both** match `rs1 = x5`. The architecture says the consumer reads the value written by the *most recent preceding* write — 101. The instruction in MEM is the newer writer, so EX/MEM must win. Testing MEM/WB first would resurrect the stale 100.

The RTL expresses this as ordering in a conditional chain — `(mem_writes && ...) ? 2'b10 : (wb_writes && ...) ? 2'b01 : 2'b00` — so the priority is structural rather than a separate signal that could be got wrong.

`asm/hazard/fwd_ex_ex.s` ends with exactly that three-instruction sequence, and swapping the priority was one of the injected mutations: it was confirmed to make that test FAIL before the correct RTL was restored (DESIGN.md §10.4).

5.3 Why a load-use hazard needs a stall, and why exactly one cycle

Why forwarding cannot fix it. For every other instruction, the result exists at the end of EX, so it can be bypassed to an instruction one slot behind. A load's data does not exist until the end of **MEM** — the memory has not answered yet when the consumer needs it. There is no wire to forward from. This is the one genuinely unavoidable data hazard in a 5-stage pipeline, and it is why `EX/MEM.res` deliberately does not serve loads: for a load, `res` is the *effective address*, not the data.

The condition (`hazard_unit.v`):

```
stall = ID/EX.valid && ID/EX.mem_re && ID/EX.rd != 0 &&
      ( (ID reads rs1 && ID.rs1 == ID/EX.rd) ||
        (ID reads rs2 && ID.rs2 == ID/EX.rd) )
```

- `mem_re` — only loads.
- `rd != 0` — `lw x0, 0(rs1)` writes nothing, so nothing depends on it.
- “ID reads rsN” comes from the decoder, not the raw bits (§3.17).

Why exactly one cycle is always enough. After one stall cycle the load has moved from EX to MEM, and the consumer re-decodes in ID; the cycle after that, the consumer is in EX and the load is in WB, where the MEM/WB bypass (`2'b01`) supplies real data. One cycle of delay converts a distance-1 dependency into a distance-2 dependency, and distance 2 is covered by a bypass that already exists. There is no case that needs two.

		c1	c2	c3	c4	c5	c6	c7
lw	x7,0(x5)	IF	ID	EX	MEM	WB		
add	x8,x7,x0		IF	ID	ID	EX	MEM	WB
						+-	lw in WB: MEM/WB forward (2'b01)	
						+	----- add re-decodes; ID/EX got a bubble	
						+	----- hazard detected: stall raised	
sub	...		IF	IF	ID	EX	MEM	
					^^	PC and IF/ID held, imem en = 0		

What a stall physically does — four things in the same cycle, and all four are necessary:

1. `pc.v`'s enable goes low (the PC holds);
2. `imem.v`'s enable goes low — *this is the one people forget*: the instruction word lives in IMEM's output register, so holding only the PC would let the next instruction overwrite it;
3. the IF/ID register holds;
4. the ID/EX register is loaded with a **bubble** — which is what turns a stall at the front into a gap travelling down the back of the pipe.

`asm/hazard/load_use.s` covers every shape: the loaded value used as `rs1`, as `rs2`, as both at once, as a store's base address, as a store's data, and as a branch operand.

5.4 Why `jal` costs 1 bubble but `branch` and `jalr` cost 2

The cost of a control transfer is the number of instructions already fetched behind it when it resolves.

Redirect	Resolved in	Why there	Flushes	Cost
<code>jal</code>	ID	Target is <code>PC + immJ</code> — no register value needed, and both are available in ID	IF/ID	1 bubble
conditional branch	EX	The condition needs <code>rs1/rs2</code> , which must be <i>forwarded</i> , and forwarding happens in EX	IF/ID + ID/EX	2 bubbles
<code>jalr</code>	EX	Target is <code>(rs1 + immI) & ~1</code> — needs a register value and an adder	IF/ID + ID/EX	2 bubbles
<code>ecall</code> trap	EX	Where the trap is attached to a valid instruction	IF/ID + ID/EX + EX/MEM	2 bubbles
<code>mret</code>	EX	Reads <code>mepc</code> from the CSR file, which lives in EX	IF/ID + ID/EX	2 bubbles

Why not resolve branches in ID too? It is possible — some designs do — but it would require the comparator *and* the forwarding network in ID, which means moving the bypass muxes one stage earlier and adding a new hazard: a branch whose operand is produced by the immediately preceding instruction could not be forwarded at all in ID (the producer is only in EX), so it would need a stall that the EX-resolved version does not. The project spec fixed the placement explicitly (PROJECT-REQUIREMENTS.md §3.2: branches and `jalr` in EX, `jal` in ID), and the trade is: 2 bubbles per taken branch instead of 1, against a shorter ID stage and no new stall class. The branch predictor then buys back the difference on predictable branches (§7.1).

Why `jal` flushes only IF/ID. The `jal` is itself in ID and must continue into EX to compute and write its link value `PC+4`. Flushing ID/EX would destroy the `jal` itself. Only the instruction behind it (in IF) is on the wrong path.

Why `ecall` also flushes EX/MEM. Because the `ecall` must *not* retire: it is the trapping instruction, `mepc` points at it, and it will re-execute after the handler returns. `cpu_top.v` does this with `flush_ex = trap_taken & ~halt` on the EX/MEM register. `mret`, by contrast, *does* retire — it is a completed instruction, not a squashed one.

5.5 Why store data (`rs2`) is forwarded

A store reads two registers: `rs1` (base address, goes through the ALU) and `rs2` (the data to store, which bypasses the ALU entirely and goes to `dmem.wdata`). `rs2` is just as likely to be produced by the immediately preceding instruction:

```
addi x6, x0, 0x1234
sw   x6, 0(x5)      # x6 must be forwarded into the store data path
```

Without `fwd_c`, this would store whatever stale value the register file held. The path is separate from `fwd_b` — even though the two compute the same function — so that it is visible in the datapath and can be tested independently by `asm/hazard/store_data_fwd.s`. The store-data forward is on the project’s non-negotiable correctness checklist (CLAUDE.md) precisely because it is the bypass most often left out: the ALU operands are obvious, the store data is not.

5.6 Why the WB→ID internal bypass exists

Hazard distance 3: the producer is in WB, writing the register file *this* cycle, while the consumer is in ID reading it. The write lands at the end of the cycle; the read happens during it. Without intervention the consumer reads the stale value.

Two standard fixes exist:

- **Write on the negative clock edge**, so the write completes at mid-cycle, before the read settles. This works but makes the design dual-edge, which complicates static timing analysis and is generally discouraged in FPGA flows.
- **An internal bypass in the read path** — what this design does: `(we && waddr == raddr) ? wdata : regs[raddr]`.

The bypass keeps everything on one clock edge. Its cost is one comparator and one mux per read port, inside the register file where the write data is already present. Removing it was an injected mutation and `tb_regfile` caught it (DESIGN.md §10.4); the `asm/hazard/fwd_wb_id.s` program exercises the same distance at the program level.

5.7 The interaction that had to be fixed: ID-stage redirects during a stall

This is the most interesting bug in the design and the best thing to have ready if asked “what was hard?”.

The rule everywhere else: a redirect overrides a stall, because the stalling instruction is being killed anyway. This is expressed three times, deliberately redundantly: `stall` is suppressed while a flush is happening; `flush` beats `stall` inside every pipeline register; and the fetch enable is `redirect | ~stall`.

Where that rule breaks. A `jal` in ID, or a branch the BHT predicts taken, is a redirect raised *by the instruction in ID itself* — and that instruction is not on a wrong path, it is the one committing the machine to a new PC. If it redirected while simultaneously being stalled, the stall would bubble it out of ID/EX at the same moment the redirect reloaded IF/ID with the target. The instruction would then exist in neither register: its link-register write silently lost (`jal`), or the branch simply vanished. Note the stall need not be the ID instruction’s own fault — neither a `jal` nor a branch can raise a load-use interlock on its own account, but an unrelated older instruction can have one active on the same cycle.

The first fix, and why it stopped working. The original design added a `!redirect_id` term to `stall`, so an ID-stage redirect always suppressed its own stall. That worked until the branch predictor gave `redirect_id` a second source: `redirect_id = jal_taken | bht_taken`, where `bht_taken` derives from `id_pred_taken`, which does not depend on `stall`. A `stall` expression reading `redirect_id` would then close a combinational loop — `stall → redirect_id → stall` — with two stable states.

The fix that is in the RTL. Gate on the *redirect* side instead:

```
assign jal_taken    = if_id_valid & id_jal & ~stall & ~ebreak_pending;
assign bht_taken    = id_pred_taken      & ~stall & ~ebreak_pending;
assign redirect_id = jal_taken | bht_taken;
```

`redirect_id` is now structurally zero on any cycle `stall` is asserted, without either signal referencing the other, and `hazard_unit.v`’s `stall` output no longer mentions `redirect_id` at all. **The rule is: an ID-stage redirect only fires in the cycle its own instruction actually advances.** A stalled `jal` or a stalled predicted-taken branch does not lose anything — it redirects one cycle later. Maximum cost: one cycle.

The payoff, beyond fixing the loop. Under the old rule, a mutation that over-approximates the load-use interlock (removing the `id_uses_rs*` qualifiers) actually *broke* bloop, losing one retired instruction per loop iteration — a spuriously stalled `jal` redirected and was bubbled out of ID/EX in the same cycle. Under the current rule the same mutation is harmless to correctness and costs only cycles (DESIGN.md §10.4). The design became robust to a whole class of stall over-approximation, not just to the specific case.

5.8 The FORWARDING = 0 build — how the comparison is made fair

FORWARDING is a compile-time parameter on `cpu_top`, threaded into both `forward_unit.v` and `hazard_unit.v`. At 0:

- every forwarding select is forced to `2'b00`, so an operand can only come from the register file;

- correctness is restored by stalling instead: ID waits while *any* pending write to one of its source registers is still in EX or MEM.

A producer in EX costs 2 stall cycles, a producer in MEM costs 1, and a producer in WB costs 0 because the register file's WB→ID bypass still covers it. Loads need no special case — the general rule already holds the consumer until the load reaches WB.

Why this is the right baseline. It is the *same RTL*, the same programs, the same testbenches, with one parameter changed. Every test in `asm/insn`, `asm/hazard`, `asm/prog` and `tb/bringup` passes under both builds, so the two machines are provably functionally identical and differ only in cycle count. That is what makes the CPI comparison in §6.1 an honest measurement of forwarding rather than a comparison of two different designs.

6. Performance and resources — every number, and why it is that number

6.1 CPI: forwarding on versus off

This is the course's “show the performance of your CPU” deliverable. Both columns are the same RTL, the same programs and the same testbenches, with `FORWARDING` changed. BHT off throughout, so this table isolates forwarding. (Source: `DESIGN.md` §11.2.)

Program	Insns	Cycles			Cycles			Speedup
		fwd=1	CPI	lu_stalls	fwd=0	CPI	Stalls	
<code>fib</code>	658	900	1.368	62	1489	2.263	651	1.65×
<code>bsort</code>	1366	1880	1.376	136	2601	1.904	857	1.38×
<code>bloop</code>	2878	4019	1.396	0	4913	1.707	894	1.22×
total	4902	6799	1.387	198	9003	1.837	2402	1.32×

Where the residual 0.387 CPI comes from. Ideal CPI is 1.0. At `fwd=1` there are only two sources of extra cycles in the entire machine — load-use stalls and control-flow bubbles — and they account for the gap exactly. Worked example on `fib`, which is the cleanest case because it contains **no `jal` or `jalr` at all** (its three loops all use conditional backward branches; verified by inspecting `asm/prog/fib.s`):

```

pipeline fill (4 cycles before the first instruction retires)    4
retired instructions, one per cycle                             658
load-use stall cycles (measured, perf_lu_stalls)                62
taken branches x 2 bubbles:  91 branches, static-NT accuracy
3.3 % -> 88 taken, 88 x 2                                     176
-----
total                                                            900  = measured

```

The arithmetic closes exactly on the measured 900 cycles. For `bsort` and `bloop` the same accounting leaves a residual (202 and 645 cycles respectively) which is the `jal/jalr` redirects those programs do contain — `bloop` uses `j` for all three of its loop back-edges, at 1 bubble each, which is why its overhead is large even though its `lu_stalls` count is zero.

Why `fib` gains the most from forwarding (1.65×). Its inner loop is a tight dependent chain: each iteration's addition consumes the previous iteration's result almost immediately. Every one of those RAW hazards is a stall at `fwd=0` and free at `fwd=1`.

Why `bloop` gains the least (1.22×) despite having the most instructions. Two reasons that compound. Its `lu_stalls` count is **zero even at `fwd=1`** — its loop bodies compute independent quantities, so it was never paying the one hazard forwarding actually removes. And its `fwd=0` penalty (894 stall cycles) is pure ordinary-RAW stall. `bloop` shows what forwarding is *for*, in the negative: a program with no producer-consumer adjacency has nothing for the bypass network to save.

Across the other suites: the 9 hazard programs are CPI 1.429 vs 2.044; the 46 per-instruction programs are 1.269 vs 1.369.

6.2 Resources: 1461 LUTs, 819 flip-flops, 2 BRAMs

All post-place-and-route, out-of-context, xc7a35tcpg236-1, from `vivado/reports/utilization*.txt`.

Design	Constraint	LUT	of which LUT-RAM	FF	BRAM
Base (BHT_ENABLE=0)	10.0 ns	1245	44	689	2
Full (BHT + interrupt)	10.0 ns	1461	44	819	2
Full	12.5 ns	1455	44	819	2
Full	13.5 ns	1424	44	819	2

Why the LUT count varies with the constraint but the FF count does not. Flip-flops are architectural state — the RTL declares them and no timing constraint changes how many exist. LUTs are *implementation*: a tighter constraint makes Vivado duplicate logic to shorten paths (replication to reduce fanout) and a looser one lets it share more. The 1424–1461 spread across three constraints is $\pm 1.3\%$, which is the normal noise of that process, not a design difference.

Flip-flop budget. The RTL declares 1052 bits of state:

Holder	Bits	Note
<code>pc.pc_q</code>	32	
<code>imem.inst</code>	32	Serves as the IF/ID instruction register
<code>if_id</code>	35	pc 32 + valid 1 + pred_state 2
<code>bht.ctr</code>	128	64×2
<code>id_ex</code>	212	The PW localparam
<code>ex_mem</code>	144	
<code>dmem read regs</code>	37	rword 32 + addr low 2 + funct3 3
<code>csr</code>	99	mtvec 32 + mepc 32 + mcause 32 + MIE + MPIE + MEIE
<code>mem_wb</code>	140	
<code>perf_counters</code>	192	6×32
<code>done_r</code>	1	
total declared	1052	

Post-route reports **819**, so synthesis removed 233 bits. Bits that are provably constant are the obvious ones — `mtvec[1:0]` and `mepc[1:0]` are literal `2'b00` — and the rest is Vivado merging registers that are exact delayed copies with no independent fan-out. The useful cross-check is that the base machine declares 921 bits (1052 minus the BHT's 128, IF/ID's 2 prediction bits and ID/EX's 1) and reports 689, a trim of 232 — the same size. The trimming is a property of the datapath, not of the predictor.

The 64 “set” registers. `utilization.txt` §1.1 splits the 819 into 755 with a synchronous **reset** and 64 with a synchronous **set**. Those 64 are the low bits of the BHT counters: `reset` puts every entry in `2'b01`, so `ctr[i][0]` resets to 1 (an FDSE, set) and `ctr[i][1]` resets to 0 (an FDRE, reset). `hold.txt` confirms it directly — one of the listed paths ends at `u_bht/ctr_reg[0][0]/S`. This is a satisfying detail to be able to explain: an examiner asking “why are 64 of your registers set rather than reset?” is asking about the BHT's reset-to-weakly-not-taken policy, from the other end.

Cost of the bonus features. Full minus base = **+216 LUT**, **+130 FF** for the branch predictor (128 counter bits + 2 bits of prediction carried in IF/ID, plus the saturating update logic, the ID redirect path and the EX mispredict comparison). The interrupt/CSR subsystem is present in both builds, so its cost is not isolated by this pair; it is roughly the 99 declared CSR bits plus the trap logic in EX.

6.3 What maps to what — and the ram_style story

`vivado/reports/memory.txt` is a `get_cells -hierarchical` dump of every memory primitive in the routed netlist. It exists because “the memories are in block RAM” is the kind of claim that is easy to assume and easy to get wrong.

Array	Size	Maps to	Evidence in <code>memory.txt</code>
<code>u_imem/mem</code>	1024 × 32 ROM	1 × RAMB36E1	<code>u_imem/inst_reg</code>
<code>u_dmem/mem</code>	1024 × 32 RAM	1 × RAMB36E1	<code>u_dmem/mem_reg</code>
<code>u_regfile/regs</code>	32 × 32, 2 async read ports	distributed LUT RAM, 44 LUTs	20 × RAMS32 + 68 × RAMD32 under <code>u_regfile/</code>

On the register file’s primitive count: the post-route dump lists 88 RAM32 primitives (20 single-port RAMS32 + 68 dual-port RAMD32), which pack two-per-LUT6 into the 44 LUTs `utilization.txt` reports as “LUT as Distributed RAM ... using O5 and O6 = 44”. `DESIGN.md` §11.5 quotes the synthesis-stage Final Mapping Report’s view of the same thing as “12 × RAM32M”; both describe one array and the authoritative number is **44 LUTs**.

Why the register file is LUT RAM but the memories are block RAM. This is a good question to be asked, because the answer is not “small things go in LUTs”:

- The register file needs **two asynchronous read ports**. Distributed RAM provides exactly that — a LUT’s read path is combinational. Block RAM cannot: a BRAM read is clocked, so putting the register file in a BRAM would force a registered read and change the entire ID stage and forwarding structure.
- The memories need **capacity and a single synchronous port each**, which is precisely what a BRAM is. 1024 × 32 in LUTs would cost thousands of LUTs; in a BRAM it costs 1 of 50.

So the split follows from the *access pattern*, not the size.

Why `ram_style` attributes are not optional. Left to itself, Vivado made the opposite choice on all three arrays:

- it **constant-folded the instruction ROM into LUT logic and dropped it from the netlist entirely**, because the image loaded at synthesis was sparse;
- it put the data memory in 512 RAMS64E distributed-RAM cells;
- it packed the register file — the one array that genuinely wants LUT RAM — into the design’s only block RAM.

`rtl/imem.v` and `rtl/dmem.v` therefore carry `(* ram_style = "block" *)` and `rtl/regfile.v` carries `(* ram_style = "distributed" *)`. Verilog-2001 attributes are ignored by xsim, so simulation is unaffected.

The “88 MHz was fake” story — say this one before being asked. Before 2026-09-07 the project reported 2173 LUT / 960 FF / 1 BRAM / WNS −1.39 ns → “fmax about 88 MHz”, and claimed IMEM was in block RAM. All of it was wrong, from one root cause: synthesis initialised IMEM with `asm/smoke.hex`, a ~60-instruction bring-up image, and with no `ram_style` attribute Vivado optimised the instruction memory out of the design altogether. The single block RAM in that netlist was the **register file**. The reported numbers characterised the loaded program, not the CPU.

Two things fixed it, and both are in the flow now: the `ram_style` attributes, and `synth.tcl` defaulting `IMEM_INIT` to `asm/prog/bpred.hex` — a real program image, because a near-empty ROM invites exactly this optimisation. The honest number came out *worse* (74 MHz instead of 88 MHz) because the block RAM’s 2.454 ns clock-to-output is most of the difference. **Mapping the memories correctly costs fmax; it also makes the number mean something.** This is recorded in `DESIGN.md` §11.5 under “Superseded measurement” rather than quietly corrected, so the change is traceable.

6.4 Why 74 MHz, and what the critical path actually is

The operating point. The full design closes setup timing at **13.5 ns = 74.07 MHz** (`timing_13.5ns.txt`: WNS +0.052 ns, TNS 0.000, 0 of 2956 failing endpoints). 100 MHz is not met (WNS −3.235 ns, 555 failing) and neither is 80 MHz (−0.755 ns, 306 failing).

Why quote 74 and not 75.6. The three constraint points agree to within 1.2 % when converted by $1/(T - WNS)$: 75.6 MHz at 10 ns, 75.4 at 12.5 ns, 74.4 at 13.5 ns. That agreement is the useful cross-check — the frequency is a property of the design, not of the constraint it was asked to meet. 74 MHz is quoted because it is the one a place-and-route run actually *demonstrated* by closing, rather than extrapolated from a run that failed.

The critical path (`timing.txt`, 10 ns run): 12.982 ns, 15 logic levels, 35 % logic / 65 % routing, from `u_dmem/mem_reg/CLKBWRCLK` to `u_pc/pc_q_reg[31]/CE`. Step by step, with the delays from the report:

#	Step	Delay	What it is
1	RAMB36E1 CLKBWRCLK → DOBDO	2.454 ns	The DMEM block RAM's clock-to-output — 19 % of the path in one hop, the largest single term
2	2 × LUT6	~0.25 ns + route	Byte-lane select and sign/zero extension of the load result (<code>dmem.v</code> , combinational on the registered word)
3	LUT6	~0.12 ns + route	The MEM/WB → EX forwarding mux , delivering the load result to a dependent branch
4	3 × CARRY4	~0.61 ns	The branch comparator's carry chain producing branch_cond
5	4 × LUT5/6	~0.5 ns	The redirect / flush / stall reduction
6	→ u_pc/pc_q_reg[31]/CE	—	The PC register's clock enable

Routing dominates (8.430 ns of the 12.982), which is normal for a design spread thinly across a small device with a 2-BRAM anchor.

Why *this* chain is the longest. It is the one case where a value has to travel from a memory, through the bypass network, through a comparison, and back into a control decision **all within one cycle**: a load result reaching the branch comparator through forwarding, and then deciding the next PC. Every other path either starts at a flip-flop (no 2.454 ns clock-to-out) or ends in a datapath register rather than in the PC's enable (fewer levels of control reduction at the end). Note that at 13.5 ns the same source drives `u_imem/inst_reg/ENARDEN` instead — the other consumer of the same stall term — for 12.921 ns over 17 levels, i.e. the same chain with a different tail.

Note what step 1 tells you about §6.3. That 2.454 ns did not exist in the old measurement, when DMEM was in distributed RAM and the read was roughly 1 ns. The 2.454 ns is the price of a correct BRAM mapping, and it is most of the difference between the fake 88 MHz and the real 74 MHz.

The two named fixes, and why they target this chain (DESIGN.md §11.5 — neither is implemented; both were out of scope under the Sep 15 feature freeze):

1. **Register the DMEM output lane-extension into WB.** Move the byte-lane selection and sign/zero extension out of the same cycle as the BRAM read, behind the MEM/WB register. This removes steps 2–3 from the critical cycle and is clearly the first fix, because it attacks the part of the path adjacent to the 2.454 ns term. It reshapes the load-use interlock, so it is a datapath change, not a local one.
2. **Pipeline the forwarding compare.** Precompute the `rs == rd` match bits in ID, where the register addresses are available a cycle earlier, instead of comparing them combinatorially in EX. This shortens step 3's control side.

A third option is specific to the memories: Vivado's **Synth 8-7052** warning says both block RAMs could absorb an optional output register if one were provided, which would turn step 1 into roughly 0.4 ns of clock-to-out — at the cost of one extra cycle of memory latency, i.e. a pipeline-depth change, and therefore a different machine.

Neither fix would change any instruction's architectural behaviour.

6.5 Two methodological points about the synthesis numbers

Out-of-context synthesis. `cpu_top` has 363 ports, 361 bits of which are commit-trace and performance-counter observation outputs. The `cpg236` package offers 106 I/O pins, so the design *cannot* be pinned out as a top-level: synthesis runs with `-mode out_of_context`. That means no I/O buffers and no BUFG on the clock. It is also the

methodologically correct way to characterise a core that is not being placed on a board — the numbers describe the CPU's logic, not a particular pinout.

Hold timing, and why the violation is not real. `hold.txt` reports $WHS = -0.120$ ns across 483 failing endpoints at every constraint point. Hold being period-independent is the first clue about what it is: **every one of the twenty worst hold paths starts at the `rst` input port** (fanout 485, matching the 483 failing endpoints). The same analysis restricted to register-to-register paths (`hold_reg2reg*.txt`) is **MET**: +0.071 ns at 10 ns, +0.112 ns at 13.5 ns, +0.119 ns on the base machine. **There are no internal hold violations.**

The cause is the out-of-context flow, and Vivado says so in the same run: [Timing 38-242] The property `HD.CLK_SRC` of clock port "`clk`" is not set and [Route 35-198] Port "`rst`" does not have an associated `HD.PARTPIN_LOCS`. Concretely: with no BUFG, the destination registers see the clock after 0.973 ns of general routing, while `rst` is declared to arrive with 0 ns input delay — so the tool sees data arriving 0.973 ns “early” relative to a clock edge that a real global buffer would have delayed identically at both ends. In a top-level design with the clock on a BUFG the check disappears.

This is reported rather than omitted deliberately: quoting WNS and saying nothing about WHS is exactly what lets a real hold violation go unexamined.

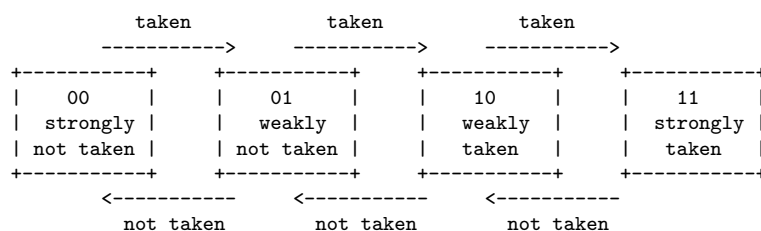
Port constraints. `constraints.xdc` declares the 361 trace/perf bits as false paths (so they cannot inflate or mask the real critical path) and gives them a 0 ns output delay (because Vivado's `check_timing` counts a port as constrained only if it carries an actual delay object). `rst`, `irq` and `done` are real I/O and are constrained with 0 ns external delay — out-of-context synthesis has no board model to reference, so 0 ns is the neutral choice. The result is a clean `check_timing`: 0 unconstrained inputs, 0 unconstrained outputs, 0 unconstrained internal endpoints, where earlier runs reported 2 and 361.

7. The bonus items — rationale and results

The task statement offers three bonus items: interrupt, cache, branch prediction. Two are implemented and measured; the third is designed and deliberately not built. That decision is the one most worth being able to defend, so it gets its own subsection.

7.1 Bonus A — the 2-bit branch history table

The four states and the FSM



00 saturates downward, 11 saturates upward
 prediction = state[1]: 00, 01 -> not taken ; 10, 11 -> taken
 reset state = 01 (weakly not taken)

Why 2 bits and not 1 or 3

Against 1 bit (remember the last outcome): a loop executed n times mispredicts **twice** per execution of that loop — once on entry, because the bit still remembers the previous execution's exit, and once on the exit itself. A 2-bit counter needs two consecutive contrary outcomes to change its prediction, so a single not-taken exit moves 11 → 10 without changing the prediction. One mispredict per loop instead of two. For a loop nested inside another loop, that halving applies on every pass of the outer loop — which is exactly the structure of `bpred.s`, and is why it reaches 95.5 % accuracy.

Against 3 bits: more hysteresis helps only on branches that are already extremely stable (where a 2-bit counter is at 93–99 % anyway) and *hurts* on branches that genuinely change behaviour, because it takes three contrary

outcomes to react. It costs 50 % more storage (192 bits rather than 128). And it would not touch the workload where this predictor actually loses: `bloop`'s alternating branch mispredicts 100 % of the time under *any* saturating counter, of any width, because the pattern has no bias to learn.

Why reset to 01 (weakly not taken)

This makes an unvisited branch behave **exactly like the static not-taken machine**. The consequence is a guarantee worth stating: *turning the predictor on can never make a first encounter with a branch worse*. Reset to 00 would behave the same on the first encounter but take two taken outcomes to start predicting taken, so a loop would warm up more slowly; reset to 10 would predict taken on every branch's first execution, which is wrong for the majority of forward branches (loop exits, `if` guards) and would make the predictor actively harmful on straight-line code. 01 also lets a loop's backward branch reach "taken" after a **single** observation.

Where each step happens

Step	Stage	Detail
Lookup	IF	Indexed by the PC the instruction memory is reading, in parallel with the fetch — off the critical path
Carry	IF/ID	The 2-bit state travels with the instruction; ID needs no second table read
Act	ID	If the instruction is a conditional branch and the state predicts taken, redirect to <code>PC + B-imm</code> — the same 1-bubble path <code>jal</code> uses
Resolve	EX	<code>mispredict = predicted_taken != actual_taken</code> ; redirect to the correct target and flush 2
Update	EX	Indexed by the EX-stage PC, with the resolved outcome, saturating

The cost table — and the asymmetry to state plainly

Prediction	Outcome	Bubbles	Why
not taken	not taken	0	Nothing is redirected; the fall-through was already being fetched
taken	taken	1	The ID-stage redirect kills the one instruction behind the branch
not taken	taken	2	The EX-stage redirect kills the two instructions in ID and IF
taken	not taken	2	The ID redirect kills one instruction; then the EX redirect kills the wrongly fetched target and refetches the fall-through

The asymmetry: this predictor can only *win* on branches that are actually taken, because a correctly predicted not-taken branch already cost nothing under static not-taken. On a workload whose conditional branches are mostly loop **exits** — not taken almost every time — the BHT has nothing to gain and a mispredict to lose. Being able to say that before being asked is worth more than the best-case number.

And note the 1, not 0. A correctly predicted taken branch still costs one bubble because there is no branch target buffer: the redirect happens in ID, after the fall-through has already been fetched. A BTB would make it zero, at the cost of a tagged CAM (§3.3).

Measured results

(FORWARDING = 1 throughout; source DESIGN.md §11.3.)

Program	Cond. branches	Static-NT acc.	Cycles BHT off	Misses	BHT acc.	Cycles BHT on	Δ
bpred	1675	14.5 %	8921	76	95.5 %	7625	−1296 (−14.5 %)
fib	91	3.3 %	900	6	93.4 %	821	−79 (−8.8 %)
bsort	288	70.1 %	1880	71	75.3 %	1890	+10 (+0.5 %)
bloop	891	72.4 %	4019	446	49.9 %	4419	+400 (+10.0 %)
irq_demo	201	99.5 %	849	1	99.5 %	849	0

bpred is the demonstration. It was written for this: nested counted loops, bit-count, linear search and triangular sum, every loop back-edge a conditional branch, 1675 dynamic conditional branches, 86 % of them taken. Static not-taken gets 14.5 % of those right; the BHT gets 95.5 %.

bloop is the counter-example, and it is adversarial by construction. Its header comment says so. Its inner `beq` tests `k & 1`, so it alternates taken/not-taken on every single iteration — precisely the pattern a saturating counter cannot learn: the counter oscillates between 01 and 10 and mispredicts **100 %** of the time. That branch runs 400 times. Its other 491 conditional branches are loop *exits*, not taken until the last pass, which the predictor gets right and which static not-taken already got right for free. And all three of its loop back-edges are `j (jal)`, so the predictor has no back-edge to win on at all.

The arithmetic closes exactly, which is why this is a good story rather than an embarrassment:

```
static not-taken: 246 taken branches x 2 bubbles      = 492 penalty cycles
with the BHT:    446 mispredicts x 2 + 0 correct-taken x 1 = 892 penalty cycles
difference       = 400 cycles
measured:       4419 - 4019                          = 400 cycles
```

The honest summary to give: a 2-bit BHT is a cheap heuristic, not a guarantee. It helps loop-dominated code (**bpred** −14.5 %, **fib** −8.8 %), is neutral on exit-dominated code (**bsort** +0.5 %), and hurts on alternating branches (**bloop** +10 %). Reporting the program where it loses is more useful than hiding it — and it demonstrates understanding of *why* it loses, which is the actual examinable content.

The predictor cannot break the machine

A mutation makes this concrete. Training the counters with the **inverted** outcome (`update_taken = ~branch_cond`) collapses accuracy — **fib** 93.4 % → 3.3 %, **bsort** 75.3 % → 25.3 %, **bloop** 49.9 % → 27.9 %, **irq_demo** 99.5 % → 1.0 % — and costs cycles (**bloop** 4419 → 4857), yet **all four programs still pass their byte-exact commit-trace diff**.

That is the correct outcome, not a hole in the test suite. A prediction only changes *when* instructions are fetched, never what they compute; correctness rests entirely on the EX-stage resolution and flush. `BHT_ENABLE = 0` independently confirms the same thing from the other side: it reproduces the base machine’s cycle counts *exactly* (4019 / 1880 / 900 on **bloop** / **bsort** / **fib**).

`tb/tb_bht.v` unit-tests the module standalone with 2600 checks: reset state, saturation at both ends, the `state[1]` prediction boundary, index aliasing (`pc` vs `pc+256` vs `pc+4`), same-cycle read/write ordering, `ENABLE = 0` inertness, and 600 randomised update/lookup steps against a behavioural model.

7.2 Bonus B — interrupts, CSRs and mret

Which CSRs, and why the set is this small

CSR	Address	Implemented	Needed for
mstatus	0x300	MIE (bit 3), MPIE (bit 7)	Enabling interrupts; saving/restoring the enable across a trap
mie	0x304	MEIE (bit 11)	Enabling the external interrupt source specifically
mtvec	0x305	BASE [31:2], mode 00	Where the handler is
mepc	0x341	[31:2]	Where to return to
mcause	0x342	all 32 bits	Why we trapped

Everything else reads zero and ignores writes — in both the RTL **and** the reference simulator, so the two agree on the behaviour of an unimplemented CSR.

Why **mtval**, **mscratch** and **mip** are dropped:

- **mtval** holds trap-specific information: the faulting address for a memory fault, the offending instruction for an illegal-instruction trap. This machine raises neither of those traps — misaligned access is don't-care (§3.15) and an illegal instruction executes as a NOP (§3.5) — so **mtval** would have nothing to report for either of the two traps that do exist (**ecall** and the external interrupt carry no address or value). 32 flip-flops storing a defined zero.
- **mscratch** is a scratch word for handlers that need a register before they can save one — the standard trick is **csrrw sp, mscratch, sp** to swap in a handler stack. Our ISR runs in the same address space with a valid **sp** already, and saves its two scratch registers on the ordinary stack. It is a software convenience, not an architectural requirement.
- **mip** reports *pending* interrupts. With exactly one interrupt source, “is an interrupt pending?” is the **irq** input line itself, which the hardware already consults directly (**irq_pending = irq & MIE & MEIE** inside **csr.v**). A readable **mip** would let software poll — but the demonstration takes the interrupt rather than polling for it.

All three are documented as “designed, not implemented” rather than ignored, which is the same posture taken with the cache (§7.3): say what was considered and why it was cut.

Interrupt entry, step by step

The trap is taken at **EX**, attached to a valid instruction, so **mepc** is always a real program counter and never a bubble's zero (**cpu_top.v: irq_taken = ex_valid & csr_irq_pending & ~ebreak_pending**).

1. the instruction in EX is **squashed** and does not retire (**flush_ex** on EX/MEM);
2. **mepc** ← PC of that squashed instruction;
3. **mcause** ← 0x8000000B (bit 31 = interrupt, code 11 = machine external);
4. **MPIE** ← **MIE**, then **MIE** ← 0;
5. **PC** ← **mtvec**.

	c1	c2	c3	c4	c5	c6	c7	
I(n-2)	EX	MEM	WB					retires normally
I(n-1)	ID	EX	MEM	WB				retires normally
I(n)	IF	ID	EX	--				SQUASHED
I(n+1)		IF	ID	--				killed (flush_id)
I(n+2)			IF	--				killed (flush_if)
ISR[0] (mtvec)				IF	ID	EX	MEM	
				~				
				c3: irq sampled with a valid instruction in EX				

Three consequences, each asserted directly by **tb/tb_irq.v**:

- **I(n)** is **cancelled, not delayed**. It never reaches memory or the register file and does not appear in the commit trace. It runs for the first time after **mret** returns.

- **I(n-1) and I(n-2) do retire.** They are older and already past EX. So up to two more instructions commit after the trap fires and before the handler’s first instruction does — which is exactly why the reference-model alignment of §8.4 is needed.
- **The interrupt outranks an ecall** in the same EX slot. `mepc` then points at the `ecall`, which re-executes after the handler returns. The reference simulator samples `irq` at the instruction boundary before decoding and makes the same choice; this is the standard RISC-V ordering.

The ecall-versus-interrupt mepc asymmetry

This is the single most common trap-handling bug and the question most likely to be asked about the interrupt bonus.

	ecall (synchronous exception)	External interrupt (asynchronous)
What <code>mepc</code> points at	The ecall itself — the instruction that faulted	The instruction that was about to execute and was cancelled
Has that instruction executed?	Yes, it reached EX and did its job (raising the trap)	No — it was squashed before doing anything
What the ISR must do	<code>mepc += 4</code> before <code>mret</code>	Nothing — leave <code>mepc</code> alone
If the ISR gets it wrong	Returns to the same <code>ecall</code> → traps again → infinite loop	Skips an instruction that never executed → silent corruption

The underlying rule is one sentence: **`mepc` always points at the instruction to resume at, and “resume” means re-execute for an interrupt but continue-past for an ecall, because the ecall already did what it was going to do.** In both cases the hardware stores the same thing — the PC of the instruction in EX — and the difference lives entirely in software. That is deliberate: the hardware cannot know whether the handler intends to retry or to continue.

`asm/prog/irq_demo.s`’s ISR does not advance `mepc` (it handles an interrupt). An `ecall` handler in the same style would need the `+4`. Both behaviours are modelled identically in `tools/iss.py`, so a mistake in either direction shows up as a trace diff.

`mret` completes the round trip: `PC ← mepc`, `MIE ← MPiE`, `MPiE ← 1`. It *does* retire (unlike the squashed trapping instruction) and it costs 2 bubbles like any other EX-resolved redirect.

Why mtvec direct mode

In vectored mode the hardware computes `mtvec.BASE + 4 × cause`, so different causes land on different entries. With exactly one interrupt source plus `ecall`, a vector table would have one used entry and a great deal of padding — and it would put a shift-and-add on the trap path in EX, feeding the PC mux that already terminates the critical path (§6.4). Direct mode makes the trap target a single input to that mux. `mtvec[1:0]` is hardwired 00, which is the architecturally correct way to advertise “direct mode only”.

What the demonstration shows

`asm/prog/irq_demo.s` installs a handler in `mtvec`, enables `mie.MEIE` and `mstatus.MIE`, and runs a 200-iteration counting loop. The ISR pushes two scratch registers, increments a visit counter, pops them and returns with `mret`. It is written so that the architectural result does **not** depend on when the interrupts land: the loop counter reaches 200 and the visit counter equals the number of interrupts taken, whatever the timing.

```

IRQ_TRAP    cycle=201  squashed_pc=0000003c  mtvec=0000004c  mie=1  mpie=0
IRQ_ENTERED mepc=0000003c  mcause=8000000b  mie=0  mpie=1
IRQ_TAKEN   retire_index=153
MRET        cycle=211  pc=00000068 -> mepc=0000003c
MRET_DONE   mie=1  mpie=1

```

Three properties demonstrated beyond the basic round trip:

- **A level, not a pulse.** An interrupt raised while `MIE = 0` (i.e. while the first ISR is still running) is **held, not lost**: it is taken 3 cycles after the pending `mret` restores `MIE`. The run with interrupts at cycles 200 and 205 shows exactly this.

- **Zero cost when unused.** An `irq` held high for an entire run of `fib`, `bsort` or `bloop` — none of which ever sets `MIE` — produces `taken = 0`, leaves `mcause` at its reset value, and gives cycle counts **identical** to runs without it. The qualification happens inside `csr.v`, so a program that does not opt in is provably unaffected.
- **Final state is timing-independent:** `x10 = 200` (loop counter), `x11 = 2` (ISR visit counter).

`tb/tb_irq.v` adds 22 structural assertions over the whole trap (§8.5), and the program is also trace-diffed against the ISS — 633 lines, zero differences, at every combination of `FORWARDING` and `BHT_ENABLE`.

7.3 Bonus C — the instruction cache: designed, not built

The design that was on the table:

Parameter	Value
Organisation	direct-mapped
Lines	128
Line size	16 B (4 instructions)
Capacity	2 KB
Index	<code>pc[10:4]</code>
Tag	<code>pc[31:11]</code>
Write policy	none needed — instruction fetch is read-only
Miss handling	stall IF, fetch four words from main memory, fill, replay

It was cut, for two reasons — and the reasons are the defensible part.

1. There is nothing to measure. The cache would sit in front of a memory that already answers in one cycle. A hit and a miss would cost **the same**, so CPI would not move at all and the hit-rate counter would be the only observable output. Making it meaningful requires a multi-cycle main-memory model — a second memory subsystem, a stall path through IF, and a refill state machine — which is a larger change than the cache itself, and it would change the baseline every other number in this project is measured against.

2. The measurement would be trivial anyway. Instruction memory is 4 KB and the cache would be 2 KB, but the largest test program is **252 bytes** of static code (`bpred.s`, 63 instructions; the others are 100–176 bytes — measured by counting non-zero words in the `.hex` images). So the entire working set of every program fits in **one eighth** of the proposed cache, and after the first pass the hit rate would be ~99–100 % on every program. **A figure that reads 99 % on every program says nothing about the design.**

The engineering judgement, stated as such: a working, measured branch predictor and a working, verified interrupt are worth more than a third bonus whose headline figure would be an artefact of the test setup. The analysis is delivered even though the hardware is not.

And the design is not closed off. The pipeline is structured so the cache could be added without touching the datapath: `imem.v` already presents a synchronous-read interface with an enable, and IF already has a stall path (the load-use interlock uses it every time it fires). A refill state machine would drive the same `en` and the same `stall`.

8. Verification methodology — and why each choice was made

8.1 Why write our own assembler and golden ISS

The course names `Mars4_5` as the simulation tool. Mars executes **MIPS**. It cannot assemble, disassemble or simulate a single RISC-V instruction — it is not a matter of configuration, it is a different ISA family. Choosing RISC-V therefore meant the project had no ready-made oracle, and two were written:

- **tools/asm.py** — a two-pass RV32I assembler. Pass 1 collects labels and `.equ` constants and *sizes* every instruction (necessary because `li` expands to one instruction when its constant fits in 12 signed bits and two — `lui + addi` — otherwise, and `la` always expands to two); pass 2 emits the encoded words. It implements all

six formats, the directives the test programs need (`.text`, `.data`, `.equ`, `.space`, `.align`) and the pseudo-ops used throughout `asm/`. Output is exactly 1024 lowercase 8-hex-digit lines — a complete `$readmemh` image of the 4 KB instruction memory.

- **tools/iss.py** — the golden reference: a second, independent implementation of the same instruction set, in Python, modelling Harvard memories, the register file with `x0` hardwired, the CSR subset of §7.2 and `trap/mret`/interrupt semantics identical to the RTL’s. Exit codes are a contract the rest of the toolchain relies on: **0** halted on `ebreak`, **2** `--max-insns` exceeded, **3** illegal instruction.

Why not Spike (the official RISC-V reference simulator)? It is a compiled C++ tool distributed as a Linux/build-from-source package, and the project’s environment rules forbid installing anything (“no Icarus, no GTKWave, no Spike”). But there is a second, better reason: **writing the reference model forces every instruction-set question the RTL has to answer — sign extension, shift-amount truncation, trap semantics, CSR write suppression — to be resolved once, explicitly, in a form that is easy to read and easy to test.** It also doubles as the assembler’s own cross-validation oracle (§8.2), which an imported binary could not do as cleanly.

The independence argument, which is the one an examiner should press on. A reference model written by the same person who wrote the RTL can share a misunderstanding with it. Three things mitigate that, and they should be said in this order:

1. `tools/test_tools.py` checks the assembler against **54 hand-verified 32-bit encodings**, computed field-by-field from the RV32I format tables *before the assembler existed*. A shared misreading of the spec cannot hide behind agreement, because these goldens do not come from either tool.
2. The ISS was written from the specification, not from the Verilog, and it is structurally different (a Python interpreter loop with no notion of pipelining), so the two are unlikely to fail identically.
3. The suite cross-validates in both directions: encode → decode round trips at every immediate-format boundary (−2048/+2047, ±4096, ±1 MiB, zero, all-ones), semantic checks, pseudo-instruction expansion, assembler error cases, and a byte-exact check of the commit-trace format through the CLI. **4323 checks**, exit 0 only if all pass.

8.2 Why the unit-test vectors are machine-generated, not hand-written

Every vector file for the unit testbenches is generated by the already cross-validated tools — `asm.encode`, `iss.decode`, or the `CONTROL` dictionary that also produces the truth table in `DESIGN.md` §3 — and **never** by a second, independent re-implementation of the same bit shuffle the RTL is being checked against.

The rule is stated explicitly in `tools/gen_imm_vectors.py` and `tools/gen_control_table.py`, and the reasoning is: a hand-rewritten immediate extractor or decode table written by the same author would only duplicate whatever they misunderstood about the spec. It would not catch the misunderstanding; it would confirm it. The independent check against the specification happens once, in the hand-verified goldens of §8.1, and everything else is generated from the artifact that check validated.

A second benefit: `gen_control_table.py --check` fails the build if the table in the design document has drifted from the dictionary that produces the test vectors. **The document and the test cannot disagree.**

8.3 Why trace-diff instead of a final-state check

A final-state check compares the 32 registers (and memory) at the end of the program. A trace diff compares **every retiring instruction**, one line each:

```
<pc (8 hex)> <insn (8 hex)> [x<rd>=<value>] [mem[<addr>]=<value>]
```

Both are used — the register file is checked too — but the trace is the primary criterion, for three reasons:

1. **Localisation.** A trace mismatch names the exact instruction at which the RTL and the reference diverged. A final-state mismatch says only “something went wrong in 2878 instructions”. For a five-stage pipeline where the bug is usually a forwarding select or a flush that fired one cycle late, that is the difference between an afternoon of debugging and a week of it.
2. **Coverage of transient state.** A value can be computed wrongly and then overwritten correctly before the program ends. A final-state check cannot see that; the trace can, because it records the write when it happens.

3. **Memory writes are visible.** `trace_mem_addr` and `trace_mem_val` put every store in the trace, so a store to the wrong address is caught at the store, not at whatever later load happens to read the corrupted location — if any ever does.

The commit-trace port is driven from the MEM/WB register outputs, qualified with `valid && !illegal`, so flushed, squashed and illegal instructions never appear. It prints exactly what `iss.py` prints, which is what makes a byte-exact diff possible (with line endings normalised, since `xsim` writes CRLF on Windows and Python writes LF).

8.4 The hard case: aligning an interrupted program against the reference model

This deserves to be raised unprompted, because it is a genuine methodological problem rather than a detail — and it shows the difference between “we ran a diff” and “we understood what we were diffing”.

The reference simulator schedules its interrupt by **retirement index**: `--irq-after N` traps at the instruction boundary once `N` instructions have retired. The RTL has no such notion — its interrupt is a *level* sampled in EX, and how many instructions have retired at that moment depends on what happened to be in MEM and WB, which in turn depends on the cycle chosen, the forwarding setting and the branch predictor. Comparing the RTL against a trace generated with a fixed `--irq-after` would be comparing **two different executions**.

So the index is **measured from the RTL rather than assumed**. When the trap fires, `tb_program` records `mtvec`; when the first instruction at `mtvec` subsequently retires, the number of trace lines already written is exactly the reference model’s `N` — every instruction older than the squashed one, and nothing younger, has committed by then. The testbench prints `IRQ_TAKEN retire_index=<N>`, and `tools/run_tests.py --irq-at` feeds those measured values back into `iss.py --irq-after` and diffs the traces byte for byte.

With that alignment, `irq_demo` matches the reference model exactly — **all 633 lines**, at every combination of `FORWARDING` and `BHT_ENABLE`. The register check uses the committed `.regs` fixture, which is timing-independent by construction.

8.5 Why mutation testing — showing the testbench fails on a broken DUT

A test that has never failed has not been shown to be a test. Before any unit testbench was accepted, it was run once against a deliberately broken copy of its DUT and confirmed to report **FAIL**; only then was the correct RTL restored. This catches the case where a testbench passes not because the DUT is right but because the testbench itself is broken — an inverted comparison, a vector file that was never regenerated, a check that silently short-circuits.

Injected bug	Where	Caught by
SRA implemented as a logical shift	<code>alu.v</code>	<code>tb_alu</code> (negative-operand vectors)
WB→ID internal bypass removed	<code>regfile.v</code>	<code>tb_regfile</code>
B-format immediate bit flipped (wrong source for <code>imm[11]</code>)	<code>imm_gen.v</code>	<code>tb_imm_gen</code>
<code>alu_ctrl</code> consults <code>inst[30]</code> for <code>addi</code>	<code>alu_ctrl.v</code>	<code>tb_control</code> (addi/sub trap goldens)
Forwarding priority swapped (MEM/WB before EX/MEM)	<code>forward_unit.v</code>	<code>asm/hazard/fwd_ex_ex.s</code>
<code>rd != 0</code> guard removed from forwarding	<code>forward_unit.v</code>	<code>asm/hazard/x0_hazard.s</code>
Load-use interlock disabled	<code>hazard_unit.v</code>	<code>asm/hazard/load_use.s</code>
1b sign extension removed	<code>dmem.v</code>	<code>tb_dmem</code> , <code>asm/insn/lb.s</code>
BHT trained with the inverted outcome	<code>bht.v</code>	<i>nothing</i> — and that is correct
Load-use <code>id_uses_rs*</code> qualifiers removed	<code>hazard_unit.v</code>	<i>nothing</i> — costs cycles only

The last two are a different kind of mutation, included deliberately: they probe **what the trace diff can and cannot detect**. Both change *when* things happen without changing *what* is computed, so both still pass every

trace diff — which is exactly the correct outcome for a check that should be blind to performance (§7.1). Being able to say “here is a class of bug our main check provably cannot catch, and here is why that is the right behaviour” is a stronger position than claiming the suite catches everything.

Note also which mutations are caught by *programs* rather than by unit testbenches: forwarding and the load-use stall only manifest across a pipeline boundary, so there is no meaningful standalone DUT to drive — `forward_unit.v` and `hazard_unit.v` are combinational functions of pipeline-register fields. Those are covered by `asm/hazard/*.s`, which is why that suite exists as a separate tier.

8.6 Why test at all four FORWARDING × BHT_ENABLE combinations

There are two compile-time parameters, so there are four machines:

FORWARDING	BHT_ENABLE	What it is
1	1	The full design
1	0	The base machine — the performance baseline
0	1	Stall-only with prediction
0	0	The simplest machine

All four must produce **identical architectural results** and differ only in cycle count. That is a strong property and it is worth testing directly, because each parameter touches logic the other could interact with: `FORWARDING = 0` changes what `hazard_unit.v`’s `stall` means, and `stall` is exactly what gates the BHT’s ID-stage redirect (§5.7 — the combinational loop lived precisely at that intersection). A bug in that interaction would be invisible at three of the four settings.

Testing all four also validates the parameters as a *measurement instrument*: the CPI comparison of §6.1 and the prediction comparison of §7.1 are only meaningful if changing a parameter provably does not change what the programs compute. It does not — every program in `asm/insn`, `asm/hazard`, `asm/prog` and `tb/bringup` passes at every setting.

8.7 The full test inventory

Tier	What	Count	Criterion
Tools	<code>test_tools.py</code> cross-validation	4323 checks	Exit 0
Unit	<code>tb_imm_gen</code>	1865 vectors	PASS
Unit	<code>tb_alu</code>	5072 vectors	PASS
Unit	<code>tb_regfile</code>	76 vectors	PASS
Unit	<code>tb_control (+ alu_ctrl)</code>	667 vectors	PASS
Unit	<code>tb_branch_unit</code>	16072 vectors	PASS
Unit	<code>tb_dmem</code>	32 vectors	PASS
Unit	<code>tb_imem</code>	10 vectors	PASS
Unit	<code>tb_pc</code>	12 vectors	PASS
Unit	<code>tb_perf_counters</code>	19 vectors	PASS
Unit	<code>tb_bht</code>	2600 checks	PASS
Directed	<code>tb_irq</code>	22 assertions	PASS at both <code>FORWARDING</code> settings
Program	<code>asm/insn/*</code> (one per encoding)	46 programs	Registers + trace exact
Program	<code>asm/hazard/*</code>	9 programs	Registers + trace exact
Program	<code>asm/prog/*</code>	5 programs	Registers + trace exact
Program	<code>tb/bringup/*</code>	7 programs	Registers + trace exact
Program	<code>asm/smoke.s</code> (all 46 encodings)	1 program	Registers + trace exact

Every testbench is **self-checking**: it prints exactly one line starting with **PASS** or **FAIL**, nothing else in the log may start a line that way, and the exit code is 0 only if a **PASS** line is present and no **FAIL** line is. There is no tier where a human looks at a waveform to decide correctness — the judgement is always in the testbench, never in the report.

The hazard programs and what each one would break (asm/hazard/, deliberately *not* NOP-padded):

Program	Hazard class	Wrong-if-broken
fwd_ex_ex.s	EX/MEM wins over MEM/WB	x11 gets the stale 100 instead of 101
fwd_mem_ex.s	Distance-2 (MEM/WB forward)	Producer-in-MEM value where the WB value is required
fwd_wb_id.s	Distance-3 (regfile WB→ID bypass)	Stale pre-write register value
load_use.s	Load-use interlock, every operand position	Garbage instead of 0x1234
store_data_fwd.s	Store-data (rs2) forwarding	Wrong word stored to memory
branch_flush.s	Flush completeness, both shadow slots	A poison addi x10, x0, 999 survives
x0_hazard.s	Never-forward-from-x0	x0 reads non-zero for one cycle
csr_hazard.s	CSR RAW visibility and rs1→CSR-source forwarding	Stale CSR value read one instruction early
mixed.s	All of the above combined	Any of the above

One detail worth knowing because it looks like a contradiction: the per-instruction programs are NOP-padded, but they **still exercise forwarding**, because `li` expands to `lui + addi` — a distance-1 RAW on the register `li` is defining. The genuinely hazard-free set is `tb/bringup/`, which is what the datapath was brought up against before any hazard logic existed.

9. Anticipated examiner questions and answers

Grouped by theme. Each answer is self-contained and one paragraph; the section reference points at the fuller argument.

9.1 ISA and encoding

Why RISC-V and not MIPS, when the course tool is Mars? Because Mars is a MIPS-only tool and cannot verify a RISC-V design at all, so the choice was between using a ready-made oracle for a weaker ISA or writing our own for a better one. RV32I has no architecturally visible delay slot, so every control hazard has to be solved in hardware rather than hidden in the instruction set — which is the point of the exercise — and its encoding keeps `rs1`, `rs2` and `rd` at fixed bit positions, which makes the ID stage cheap. Writing our own assembler and reference simulator turned out to be a benefit, not a cost: it forced every ISA semantic question to be resolved explicitly and gave us a line-by-line oracle (§1.1, §8.1).

How many instructions do you implement, exactly? 46 encodings: 37 core plus 9 system. Precisely, that is the complete RV32I base integer set *except* `fence`, plus the six Zicsr instructions and `mret` from the privileged specification. Base RV32I as the spec enumerates it is 40 instructions — our 37 plus `fence`, `ecall`, `ebreak` — so we implement 39 of 40 and add 7. The count of 46 is used consistently in every document (§1.2).

Why don't you implement `fence`? Because in this machine it would be architecturally required to do nothing, and implementing a NOP and calling it an instruction would be dishonest. `fence` orders memory operations as seen by other harts or DMA agents; this is a single-hart machine with no store buffer, no write-back cache and in-order single-cycle memory, so every memory operation is already globally ordered. `fence.i` is not even part of the base set (it is the separate Zifencei extension) and would be meaningless here anyway: IMEM is a ROM with no write path, so self-modifying code is impossible (§1.3).

Why is `wfi` illegal rather than a NOP? `wfi` means “sleep until an interrupt”, which requires a real sleep state in the fetch unit and a wake path from the interrupt logic — new states and a new class of thing to verify.

We did not implement it, so the decoder says so: `control.v` rejects `csr_addr = 0x105` explicitly. The interrupt demonstration does not need it, because taking an interrupt against a *running* instruction stream is the harder and more interesting case (§1.3).

Why is the B-type immediate's bit 0 not encoded? Because instructions are 4-byte aligned, so every branch offset is even and bit 0 is always zero. Spending an encoding bit on a known constant would be waste; dropping it and hard-wiring 1'b0 in the immediate generator doubles the reach of the same 12 encoded bits from ± 2 KiB to ± 4 KiB. The same argument gives J-type ± 1 MiB from 20 bits (§4.3).

Why is the J-type immediate scrambled? It is not arbitrary — it is the consequence of two simultaneous constraints. The sign bit must be `inst[31]` like every other format, so `imm[20]` goes there; and `imm[19:12]` must stay in `inst[19:12]`, the same physical wires U-type uses. Everything else is packed into what remains. The payoff is that `imm_gen.v` is pure wiring — a mux over fixed wire bundles with no shifters or adders anywhere (§1.4, §4.3).

Why is the CSR immediate the only zero-extended one? Because `zimm` is a 5-bit unsigned value used as a bit mask or a small constant, not a signed number. Sign-extending it would turn `csrrsi x0, mstatus, 0x18` into a write of 27 unintended ones. It reuses the `rs1` field because the immediate forms do not read a register — which is precisely why `id_uses_rs1` excludes them, so the hazard unit does not invent a dependency on a register number that is really an immediate (§3.7, §4.3).

Why 32 registers and not 16 or 64? The register field is 5 bits, and an R-type instruction needs three of them: 15 bits of register fields + 7-bit opcode + 3-bit funct3 + 7-bit funct7 = exactly 32. More registers would not fit in a word; fewer would spill to memory more often. In this design the whole file costs 44 LUTs, so the storage is not the constraint — the two asynchronous read ports are (§4.2).

9.2 Pipeline and datapath

Why five stages? It is the shallowest pipeline that still exposes all three hazard classes — structural, data and control — which is the subject of the exercise. A single-cycle machine has CPI 1.0 but a clock period covering fetch + decode + ALU + memory + write-back in one go (in our measured terms, roughly 25–30 ns), and no hazards at all. A deeper pipeline would raise `fmax` but lengthen hazard distances, deepen branch shadows (four killed instructions per taken branch instead of two) and add no new concepts (§2.1).

What happens in each stage? IF fetches from IMEM and looks up the BHT with the same PC. ID decodes the full control word, generates the immediate, reads the register file with the WB→ID bypass, and resolves `jal` and predicted-taken branches. EX forwards the three operands, computes the ALU result, evaluates the branch condition, computes branch and `jalr` targets, does the CSR read-modify-write, and takes traps and `mret`. MEM performs the byte-lane store or the synchronous load. WB selects between the result and the load data, writes the register file, and drives the commit trace (§2.2).

Why is the result mux in EX rather than WB? So that the EX/MEM forwarding source is the value the instruction will actually write back. For a `jal/jalr` that is the link address `PC+4`, not the ALU output — which is the jump target. For a `csrrw` it is the old CSR value, not the unused ALU result. If the mux were in WB, a dependent instruction would forward a jump target (§2.2, §3.14).

Why is the instruction word not in the IF/ID register? It is registered — by the instruction memory. `imem.v` is a synchronous-read memory, so its output register *is* the instruction half of the IF/ID boundary; `if_id.v` carries the PC, the valid bit and the prediction that must stay in step with it, and both are clocked by the same enable. Registering the instruction twice would add a wasted cycle of fetch latency (§2.5, §3.4).

Why in-order? Because it makes the machine verifiable and its traps precise. In-order completion means the commit trace is a total order matching the program, which is what allows a line-by-line diff against a simple sequential reference model; and it means that when a trap is taken in EX, everything older has committed and everything younger is flushed, so `mepc` unambiguously identifies the boundary. Out-of-order would need renaming, a scheduler and a reorder buffer (§2.3).

Why Harvard? A single unified memory creates a structural hazard: IF wants an instruction every cycle and MEM wants data on every load or store, and one port cannot serve both. Separate memories remove the hazard by construction, and on an FPGA they map to two independent block RAMs, so it is not even more expensive. The honest caveat is that this is a microarchitectural split visible to software — real machines get the same effect from split L1 caches over a unified address space (§2.4).

What does a “bubble” actually contain? All zeros. Every control bit zero plus `valid = 0` is a genuine architectural NOP: it cannot write a register, touch memory or redirect the PC, and it is neither retired nor traced. Reset writes the same encoding, so “after reset” and “just flushed” are the same state (§2.5).

9.3 Hazards

Why does EX/MEM take priority over MEM/WB? Because the instruction in MEM is the *newer* writer. If two older instructions both write the same register, the architecture says this instruction reads the most recent one. Testing MEM/WB first would resurrect the stale value — with `x5 = 100` then `x5 = x5 + 1`, the consumer must see 101 while 100 is still sitting in MEM/WB. `asm/hazard/fwd_ex_ex.s` is exactly that sequence, and swapping the priority was an injected mutation confirmed to make it fail (§5.2).

Why can’t forwarding solve the load-use hazard? Because the load’s data does not exist yet. Every other instruction’s result is available at the end of EX, so it can be bypassed to the next instruction; a load’s result appears only at the end of MEM. There is no wire to forward from. That is also why `EX/MEM.res` deliberately does not serve loads — for a load, `res` is the effective address, not the data (§5.3).

Why is one stall cycle always enough? Because one cycle of delay converts a distance-1 dependency into a distance-2 dependency, and distance 2 is covered by the MEM/WB bypass that already exists. After the stall, the load is in WB and its data is valid. There is no case that needs two (§5.3).

What exactly does a stall do? Four things at once: the PC holds, the *instruction memory’s enable* goes low, the IF/ID register holds, and the ID/EX register is loaded with a bubble. The second is the one people forget — the fetched instruction lives in IMEM’s output register, so holding only the PC would let the next instruction overwrite it. The fourth is what turns a stall at the front into a gap travelling down the back of the pipe (§5.3).

Why does `jal` cost one bubble but a branch costs two? Because the cost is the number of instructions already fetched behind the redirect when it resolves. `jal`’s target is `PC + immJ` — no register value needed — so it resolves in ID with one instruction behind it. A branch needs its operands *forwarded*, and forwarding happens in EX, so it resolves one stage later with two instructions behind it. `jalr` is the same: its target needs a register (§5.4).

Why doesn’t `jal` flush ID/EX as well? Because the `jal` is itself in ID and must continue into EX to compute and write its link value. Flushing ID/EX would destroy the `jal` itself. Only the instruction behind it is on the wrong path (§3.17, §5.4).

Why forward store data separately? A store reads two registers — `rs1` for the address and `rs2` for the data — and `rs2` is just as likely to have been produced by the immediately preceding instruction. Without `fwd_c`, `addi x6, ..., followed by sw x6, 0(x5)` would store a stale value. It has its own select rather than sharing `fwd_b` so the path is visible in the datapath and testable on its own (§5.5).

What is the WB→ID bypass for? Hazard distance 3: the producer is in WB writing the register file this cycle while the consumer is in ID reading it. The write lands at the end of the cycle, the read happens during it, so without the bypass the consumer reads stale data. The alternative is writing on the negative clock edge, which we rejected because it makes the design dual-edge and complicates static timing analysis (§5.6).

Was there a hazard bug you had to fix? Yes, and it is the most interesting thing in the design. An ID-stage redirect — a `jal`, or a branch the BHT predicts taken — is raised *by* the instruction in ID, and that instruction is not on a wrong path. If it redirected while also being stalled, the stall would bubble it out of ID/EX at the same moment the redirect reloaded IF/ID, so it would exist in neither register and its link write would be silently lost. The first fix put a `!redirect_id` term in `stall`, which worked until the predictor gave `redirect_id` a second source and closed a combinational loop (`stall → redirect_id → stall`) with two stable states. The fix in the RTL gates the redirect side instead: both `jal_taken` and `bht_taken` are ANDed with `~stall`, so an ID redirect only fires in the cycle its instruction actually advances. Maximum cost, one cycle (§5.7).

Are you sure forwarding is actually being exercised by your tests? Yes, in two independent ways. The `asm/hazard/` suite is deliberately *not* NOP-padded and each program targets one hazard class with a documented wrong-if-broken value. And even the NOP-padded per-instruction programs exercise it incidentally, because `li` expands to `lui + addi` — a distance-1 RAW on the register `li` is defining. The genuinely hazard-free set is `tb/bringup/`, which is what the datapath was brought up against before hazard logic existed (§8.7).

9.4 Memory

Why 4 KB each? A single RAMB36E1 is 36 Kbit, about 4.5 KB, so 4 KB is the largest power-of-two that fits in one block RAM per memory. Smaller would occupy the same primitive and waste it; 8 KB would need two BRAMs each for capacity no program uses. The largest test program is 252 bytes of static code, so 4 KB is already 16× more than needed (§4.6).

What happens on a misaligned access? It is architecturally don't-care in this implementation: no trap, the word index is `addr[11:2]` and the lane comes from `addr[1:0]`, so a misaligned `lw` reads the containing word. This is a documented simplification, not an oversight — real RV32I would raise a misaligned-access exception or emulate it. The important part is that `tools/iss.py` models the *identical* behaviour, so RTL and reference still agree bit-for-bit and the simplification cannot hide a real bug (§3.15).

Where does byte-lane handling live? Inside `dmem.v`. A store selects lanes by `addr[1:0]` and the access width; a load registers the whole word plus the address low bits and `funct3` on the MEM edge, and the lane select with sign or zero extension is combinational on those registered values, so the result is valid in WB. That is also why `funct3` has to travel all the way to MEM (§3.15).

Why is the load data not in the MEM/WB register? Because `dmem.v` already registered the memory word on the MEM clock edge. A second register would duplicate 32 flip-flops and add nothing. The cost of that choice is that the lane-select and extension logic sits in the same cycle as the 2.454 ns BRAM read, which is steps 1–2 of the critical path — and undoing it is our first named timing fix (§3.16, §6.4).

Is the memory little-endian? Yes — byte 0 occupies bits [7:0]. `lb/lh` sign-extend, `lbu/lhu` zero-extend. That is the RISC-V standard and it is what the reference model implements (§3.15).

9.5 CSRs, traps and interrupts

What is the trap asymmetry? `mepc` always points at the instruction to resume at, but “resume” means different things for the two trap kinds. For an **external interrupt**, the instruction in EX is squashed before doing anything, so `mepc` points at an instruction that has *not* executed and the ISR must **not** advance it — `mret` re-executes it. For an **ecall**, the instruction did reach EX and did its job (raising the trap), so `mepc` points at the `ecall` itself and the handler **must** do `mepc += 4` before `mret` — or it returns to the same `ecall`, traps again, and loops forever. The hardware stores the same thing in both cases (the PC of the instruction in EX); the difference lives entirely in software, deliberately, because the hardware cannot know whether the handler intends to retry or to continue (§7.2).

Why is the trap taken in EX? Because EX is where the interrupt can be attached to a *valid* instruction, which guarantees `mepc` is a real program counter and never a bubble's zero. `cpu_top.v` conditions it on `ex_valid`. Taking it in IF or ID would risk attaching it to a slot that is about to be flushed (§7.2).

Why does the interrupt outrank an ecall in the same slot? Standard RISC-V ordering: interrupts take priority over synchronous exceptions raised by the same instruction. `mepc` then points at the `ecall`, which re-executes after the handler returns. The reference model samples `irq` at the instruction boundary before decoding and makes the same choice, so the two never disagree (§7.2).

Why is `irq` a level and not a pulse? Because a level cannot be lost. An interrupt raised while `MIE = 0` — for example while the previous ISR is still running — is held and taken as soon as `mret` restores `MIE`, three cycles later, which the demonstration shows directly by asserting the second interrupt at cycle 205 while the first handler is still executing. A pulse would have to be latched somewhere, which is extra state that `mip` would normally provide (§7.2).

Why only five CSRs? Because those five are what a demonstrable external interrupt needs and no more. `mtval` would have nothing to report — the two traps we implement carry no faulting address or value, and we raise neither memory-fault nor illegal-instruction traps. `mscratch` is a software convenience for handlers that need a register before saving one; our ISR has a valid `sp` and uses the ordinary stack. `mip` reports pending interrupts, but with one source “is an interrupt pending?” is the `irq` line itself, which the hardware already consults. All three are documented as designed and dropped rather than ignored (§7.2).

Why direct-mode `mtvec`? With one interrupt source plus `ecall`, a vector table would have one used entry and a lot of padding, and it would put a shift-and-add on the trap path in EX — feeding the PC mux that already

terminates the critical path. Direct mode makes the trap target one input to that mux. `mtvec[1:0]` is hardwired 00, which is the architecturally correct way to advertise direct-mode-only (§7.2).

Do you need an interlock for CSR reads after CSR writes? No, and the reason is structural: the read-modify-write happens entirely within one EX cycle — `csr_rdata` is the pre-write value and the new value commits on the same edge — so an instruction one slot behind reaches EX a cycle later and reads the updated register. This is why `csrw mepc, t0` immediately followed by `mret` works. What *does* need forwarding is the register side: `csr_wsrc` takes the forwarded `rs1` (§3.13, §5).

What does 0x8000000B mean? Bit 31 set means “this is an interrupt, not an exception”; the low bits 0xB = 11 is the machine external interrupt code. The `ecall` cause is plain 11 with bit 31 *clear* — “environment call from M-mode”. Two different events share the number 11, distinguished by bit 31, exactly as the privileged specification defines (§4.8).

9.6 Branch prediction

Why does the BHT hurt on alternating branches? Because a saturating counter predicts the *majority* direction, and an alternating branch has no majority. `bloop`’s inner `beq` tests `k & 1`, so it flips taken/not-taken every iteration: the counter oscillates between 01 and 10 and is wrong **100** % of the time — it always predicts what just happened, which is always the opposite of what happens next. That branch runs 400 times, and the arithmetic closes exactly: 446 mispredicts $\times 2$ bubbles = 892 penalty cycles under the BHT versus 246 taken branches $\times 2$ = 492 under static not-taken, a difference of 400 cycles, matching the measured 4419 – 4019 = 400. No saturating counter of any width fixes this; it would need a history-based predictor (a two-level or gshare scheme) that can recognise the *pattern* rather than the bias (§7.1).

Then why report `bloop` at all? Because it is more instructive than the win, and because hiding the case where your feature loses is the fastest way to lose credibility. It was written deliberately to defeat a 2-bit predictor — its header comment says so — and being able to explain precisely why, with the arithmetic closing to the cycle, demonstrates understanding that the 95.5 % number on `bpred` does not (§7.1).

Why 2-bit rather than 1-bit? A 1-bit predictor mispredicts twice per loop execution: once on entry (it still remembers the previous exit) and once on the exit. A 2-bit counter needs two consecutive contrary outcomes to change its prediction, so the single exit does not flip it — one mispredict per loop instead of two. For a nested loop that halving applies on every pass of the outer loop (§4.7, §7.1).

Why reset to weakly-not-taken rather than strongly-not-taken? So that an unvisited branch behaves exactly like the static not-taken machine — turning the predictor on can never make a *first* encounter worse — while a loop’s backward branch reaches “taken” after a single observation. Resetting to 00 would warm up more slowly; 10 would predict taken on every branch’s first execution, which is wrong for most forward branches (§7.1).

Why does a correctly predicted taken branch still cost a cycle? Because there is no branch target buffer. The prediction is acted on in ID, by which time the fall-through has already been fetched, so one instruction must be killed. A BTB would make it zero, but the target of a conditional branch is `PC + B-immediate`, which ID computes for free — a BTB would be a tagged CAM storing something we can recompute. We took the one bubble and saved the CAM (§3.3, §7.1).

What if the predictor is wrong — can it corrupt the program? No, and this was tested directly: training the counters with the **inverted** outcome collapses accuracy (`fib` 93.4 % $\rightarrow$ 3.3 %) and costs cycles, yet every program still passes its byte-exact commit-trace diff. A prediction changes only *when* instructions are fetched; correctness rests entirely on the EX-stage resolution and flush. `BHT_ENABLE = 0` confirms it from the other side by reproducing the base machine’s cycle counts exactly (§7.1).

Why index with `pc[7:2]` and what about aliasing? Bits [1:0] are always zero, so the index starts at bit 2; six bits give 64 entries covering a 256-byte window. The table is untagged, so two branches 256 bytes apart share a counter and train against each other — that degrades accuracy, never correctness, and it is the standard price of a cheap predictor. For the programs measured here the effect is zero, because every test program is smaller than 256 bytes of static code, so no two branches ever collide (§4.7).

9.7 Verification

How do you know your tests aren’t trivially passing? Three independent arguments. (1) **Mutation testing:** before any unit testbench was accepted it was run against a deliberately broken copy of its DUT and confirmed to report **FAIL** — ten injected bugs are tabulated with the check that caught each (§8.5). (2) **The checks are not self-referential:** unit-test vectors are generated by the cross-validated tools, and those tools are in turn checked against 54 hand-verified encodings computed field-by-field from the format tables *before the assembler existed*, so a shared misreading cannot hide behind agreement (§8.1–8.2). (3) **The criterion is byte-exact:** not “the answer looks right” but “every retiring instruction matches an independent implementation, line for line” — 633 lines on `irq_demo`, zero differences (§8.3).

Why did you write your own reference simulator instead of using Mars? Mars executes MIPS. It cannot assemble or simulate RISC-V — it is not a configuration issue, it is a different ISA family, so there was no oracle to borrow. Spike, the official RISC-V simulator, is a build-from-source Linux tool and the project’s environment rules forbid installing anything. Writing `iss.py` also forced every instruction-set question the RTL has to answer to be resolved once, explicitly, in readable form — and it doubles as the assembler’s cross-validation oracle (§8.1).

Isn’t a reference model written by the same person unreliable? It is a real risk and it is mitigated rather than dismissed. The 54 hand-verified encodings are computed from the specification’s format tables, not from either tool, so they are an independent check on both. The ISS is structurally different from the RTL (a Python interpreter loop with no notion of pipelining), so the two are unlikely to fail identically. And the cross-check runs both directions — encode → decode round trips at every immediate boundary value — 4323 checks in total (§8.1).

Why diff traces instead of just checking the final registers? Localisation, mainly. A trace mismatch names the exact instruction where the RTL and the reference diverged; a final-state mismatch says only that something went wrong somewhere in 2878 instructions. It also catches values that are computed wrongly and then overwritten correctly, which a final-state check cannot see, and it puts every store in the record so a store to the wrong address is caught at the store rather than at some later load. We check the final register file too — but the trace is the primary criterion (§8.3).

How do you diff an interrupted program against a sequential model? Carefully, because the naive version compares two different executions. The ISS schedules its interrupt by retirement index; the RTL samples a level in EX, and how many instructions have retired at that moment depends on the cycle, the forwarding setting and the predictor. So the index is **measured from the RTL**: when the trap fires the testbench records `mtvec`, and when the first instruction at `mtvec` retires, the number of trace lines already written *is* the model’s N. That measured value is fed back into `iss.py --irq-after`. With that alignment, all 633 lines match at every parameter combination (§8.4).

Why run everything at all four parameter combinations? Because the two parameters interact at exactly the place where the hardest bug lived: `FORWARDING = 0` changes what `stall` means, and `stall` is what gates the BHT’s ID-stage redirect. A bug there would be invisible at three of the four settings. Testing all four also validates the parameters as a measurement instrument — the CPI and prediction comparisons are only meaningful if changing a parameter provably does not change what the programs compute (§8.6).

What happens on an illegal instruction? It executes as a NOP. `control.v` forces every control output to zero when `illegal` is raised, so no architectural side effect can leak out of the decoder; the instruction keeps `valid = 1` but carries an `illegal` flag, and both the retire pulse and the commit trace are qualified with `valid && !illegal`, so it is neither counted nor traced. There is no illegal-instruction trap — that would have meant a second synchronous trap cause and a second trap source in EX, which was scoped out. The behaviour is documented rather than left undefined (§3.5). One honest divergence worth volunteering: `tools/iss.py` *exits with code 3* on an illegal instruction rather than NOPing it, so the two models would disagree on a program containing one. No test program does, so the divergence never materialises — but it is a known difference, not an oversight.

What is `ebreak` doing in your instruction set? RV32I has no halt instruction, and simulation needs a defined end. `ebreak` flows through the pipeline normally, is counted and traced like any other instruction, and then raises a sticky `done` flag as it retires, which freezes the machine so nothing behind it commits and the trace ends exactly at the `ebreak` line. It is explicitly *not* a trap in this design (§1.3, §3.19).

9.8 Synthesis and timing

Why 74 MHz and not 100 MHz? Because 74 MHz is where a place-and-route run actually closed: at 13.5 ns the design meets setup with WNS +0.052 ns, TNS 0 and zero failing endpoints. At 10 ns it fails by 3.235 ns and at 12.5 ns by 0.755 ns. The three runs cross-check each other — converting by $1/(T - \text{WNS})$ gives 75.6, 75.4 and 74.4 MHz, agreeing to within 1.2 %, which says the number is a property of the design rather than of the constraint. We quote the one that closed rather than the extrapolation, and the reason it is not higher is a single combinational chain: a load result reaching the branch comparator through forwarding and then deciding the next PC, all in one cycle (§6.4).

What is the critical path, exactly, and how would you fix it? 12.982 ns over 15 logic levels, 35 % logic and 65 % routing, from `u_dmem/mem_reg/CLKBWRCLK` to `u_pc/pc_q_reg[31]/CE`. In order: the DMEM block RAM’s 2.454 ns clock-to-output (19 % of the path in one hop); the byte-lane select and sign extension of the load result; the MEM/WB → EX forwarding mux delivering that result to a dependent branch; the branch comparator’s carry chain; the redirect/flush/stall reduction; into the PC’s clock enable. The first fix is to register the DMEM lane-extension behind the MEM/WB register, which removes steps 2–3 from the same cycle as the BRAM read — it reshapes the load-use interlock, so it is a datapath change. The second is to precompute the `rs == rd` forwarding match bits in ID, where the register addresses are available a cycle earlier. Neither is implemented; both were out of scope under the feature freeze, and neither would change any instruction’s architectural behaviour (§6.4).

Why is the register file LUT RAM but the memories block RAM? Because of the *access pattern*, not the size. The register file needs two **asynchronous** read ports so that ID can read, decode and reach the ID/EX register in one cycle; a LUT’s read path is combinational, which is exactly that, while a block RAM read is clocked and would force a registered read and a different ID stage. The memories need capacity and a single synchronous port each, which is precisely what a BRAM is — and 1024×32 in LUTs would cost thousands of LUTs instead of one of the part’s fifty BRAMs (§6.3).

Did you have to force that mapping? Yes, and the story matters. Left alone Vivado made the opposite choice on all three arrays: it constant-folded the instruction ROM into LUT logic and dropped it from the netlist entirely, put the data memory in 512 distributed-RAM cells, and packed the register file into the design’s only block RAM. Before we found this, the project reported “88 MHz, 2173 LUT, 1 BRAM” — numbers that characterised the *loaded program*, not the CPU, because synthesis had been initialised with a 60-instruction bring-up image. The fix was `ram_style` attributes on all three arrays plus defaulting synthesis to a real program image. The honest number came out *worse* — 74 MHz, because the block RAM’s 2.454 ns clock-to-out is most of the difference — and that is the point: mapping the memories correctly costs fmax and makes the number mean something. The old measurement is recorded as superseded rather than quietly deleted (§6.3).

How do you know the memories really landed where you say? `vivado/reports/memory.txt` is a `get_cells -hierarchical` dump of every memory primitive in the routed netlist with its hierarchical path: two `RAMB36E1` (`u_inem/inst_reg`, `u_dmem/mem_reg`) and 88 `RAM32` primitives under `u_regfile/`, which pack two-per-LUT6 into the 44 LUTs that `utilization.txt` reports as distributed RAM. It is checked, not assumed, and it is re-checked after any RTL change to the memories (§6.3).

Why is your synthesis out-of-context? Because `cpu_top` has 363 ports, 361 bits of which are commit-trace and performance-counter observation outputs, and the `cpg236` package has 106 pins — the design physically cannot be pinned out as a top level. It is also the right way to characterise a core that is not going on a board: the numbers describe the CPU’s logic rather than a particular pinout. The caveat it carries is the hold result below (§6.5).

Your hold report shows a violation — is the design broken? No. WHS is −0.120 ns across 483 endpoints, and every one of the twenty worst paths starts at the **rst input port** (fanout 485, matching). The same analysis restricted to register-to-register paths is MET at +0.071 ns, +0.112 ns and +0.119 ns across the three runs — there are no internal hold violations. The cause is the out-of-context flow, and Vivado says so in the same run: with no BUFG the destination registers see the clock after 0.973 ns of routing while `rst` is declared to arrive with 0 ns input delay, so the tool sees data arriving “early” relative to a clock edge a real global buffer would have delayed at both ends. We report it rather than omitting it, because quoting WNS and saying nothing about WHS is exactly how a real hold violation goes unexamined (§6.5).

Why do the LUT counts differ between the 10 ns and 13.5 ns runs but the flip-flop count doesn’t? Flip-flops are architectural state — the RTL declares them and no constraint changes how many exist. LUTs are implementation: a tighter constraint makes Vivado replicate logic to shorten paths and a looser one lets it share

more. The 1424–1461 spread is ± 1.3 %, which is normal noise from that process (§6.2).

Where do 819 flip-flops come from? The RTL declares 1052 bits of state — pipeline registers 531, performance counters 192, BHT 128, CSRs 99, the PC and IMEM output register 64, DMEM read registers 37, halt 1. Synthesis removes 233 of them: bits that are provably constant (`mtvec[1:0]` and `mepc[1:0]` are literal `2'b00`) and registers that are exact delayed copies with no independent fan-out. The cross-check is that the base machine declares 921 and reports 689 — a trim of 232, the same size — so the trimming is a property of the datapath, not of the predictor (§6.2).

Why are 64 of your registers set-type rather than reset-type? They are the low bits of the BHT counters. Reset puts every entry in `2'b01`, so `ctr[i][0]` resets to 1 (an FDSE) and `ctr[i][1]` to 0 (an FDRE). It is the reset-to-weakly-not-taken policy showing up in the utilization report (§6.2).

What does the BHT cost? +216 LUT and +130 FF against the base machine — the 128 counter bits plus the 2 bits of prediction carried in IF/ID, and the LUTs for the saturating update logic, the ID-stage redirect path and the EX-stage mispredict comparison. That is 17 % more LUTs than the base machine (216/1245), bought –14.5 % cycles on `bpred` (§6.2, §7.1).

9.9 Scope, process and the awkward questions

Why no cache? Because behind a single-cycle memory a hit and a miss cost the same, so CPI would not move at all and the hit rate would be the only observable output — and that hit rate would read ~99–100 % on every program, because the largest test program is 252 bytes of static code against a proposed 2 KB cache. A figure that reads 99 % regardless of the program says nothing about the design. Making it meaningful would require a multi-cycle main-memory model — a second memory subsystem, a stall path through IF and a refill state machine — which is a bigger change than the cache itself and would move the baseline every other number in the project is measured against. The cache is designed and written up, with parameters; the judgement was that a working, measured predictor and a working, verified interrupt are worth more than a third bonus whose headline figure would be an artefact (§7.3).

Could you add it later? Yes, without touching the datapath: `imem.v` already presents a synchronous-read interface with an enable, and IF already has a stall path that the load-use interlock uses every time it fires. A refill state machine would drive the same two signals (§7.3).

Did you use Vivado 2019.2 as the brief specifies? No — 2026.1, and the RTL is deliberately kept version-portable to make that a non-issue: plain Verilog-2001, memories as `reg [31:0] mem [0:N-1]` with `$readmemh`, no IP catalog, no block design, and a `create_project.tcl` committed instead of a version-locked `.xpr`. Nothing in the design uses a feature newer than Verilog-2001 (§PROJECT-REQUIREMENTS.md §2).

What would you do differently with more time? In order: (1) register the DMEM lane-extension into WB and re-measure `fmax` — it is the clear first fix and it would attack the largest term on the critical path; (2) add a multi-cycle memory model and *then* the I-cache, so the cache has something to show; (3) add an illegal-instruction trap so the RTL and the ISS agree on that case; (4) a gshare or two-level predictor, which is the only thing that would help `bloop`'s alternating branch.

What is the weakest part of the design? The timing result. 74 MHz is honest but it is not fast, and the reason is a single chain we understand precisely and have not fixed because it landed after the feature freeze. The second-weakest is the trap surface: two trap causes and five CSRs is the minimum that demonstrates the mechanism, and a real M-mode implementation has considerably more.

What are you most confident about? That the machine computes the right answers. Every program — 46 per-instruction, 9 hazard, 5 program-level, 7 bring-up, plus the all-encodings smoke test — matches an independently written reference model byte-for-byte on every retiring instruction, at all four parameter combinations, and the testbenches themselves have been shown to fail on deliberately broken RTL.

10. Known limitations — the honest list

Volunteering these is stronger than being caught by them. Each is a deliberate, documented decision, not an unknown.

Limitation	Why it is acceptable here	Where documented
No illegal-instruction trap (illegal = NOP)	Would need a second synchronous trap cause and source in EX; scoped out. RTL and ISS diverge here (ISS exits 3), but no test program contains an illegal instruction	§3.5, DESIGN.md §5.5
Misaligned accesses are don't-care	Modelled identically in the reference simulator, so the simplification cannot mask a bug	§3.15, DESIGN.md §7
Harvard split is visible to software	Programs are loaded by <code>\$readmemh</code> , not by a loader; consistent with having no <code>fence.i</code>	§2.4
Only 5 CSRs; <code>mtval/mscratch/mip</code> dropped	Nothing in the implemented trap set needs them	§7.2
<code>mtvec</code> direct mode only	One interrupt source; vectored mode would add logic to the trap path in EX	§7.2
No I-cache	Nothing to measure behind single-cycle memory; ~99 % trivial hit rate	§7.3
No branch target buffer	Correctly-predicted-taken costs 1 bubble instead of 0	§3.3, §7.1
74 MHz, not 100	Critical path understood precisely; two fixes named, both post-freeze	§6.4
Hold shows -0.120 ns	Entirely <code>rst</code> -port paths, an out-of-context artefact; register-to-register hold is met	§6.5
Synthesis is out-of-context	363 ports vs 106 pins — it is the only option, and the right characterisation method	§6.5
No FPGA board demo	Simulation + synthesis report was the agreed scope	PROJECT-REQUIREMENTS.md §2

Appendix A — where every claim comes from

Source file	What it establishes
PROJECT-REQUIREMENTS.md	The spec, the final decisions, the corrections to the original design
docs/DESIGN.md	The living design document — truth table (§3), hazards (§6), CSRs (§8), bonus (§9), verification (§10), performance and synthesis (§11)
docs/INTERFACES.md	The binding port-level contracts between tools, RTL and testbenches
rtl/*.v (19 files, 2345 lines)	The implementation; every module's header comment states its own contract
vivado/reports/utilization*.txt	LUT / FF / BRAM, at each constraint and for the base machine
vivado/reports/timing*.txt	WNS / TNS / failing endpoints, and the full critical-path breakdown
vivado/reports/hold*.txt	WHS overall and register-to-register
vivado/reports/memory.txt	Which primitive each array actually landed in, post-route

Source file	What it establishes
<code>vivado/constraints.xdc</code>	Clock period, false paths on the observation ports, I/O delays
<code>tools/asm.py</code> , <code>tools/iss.py</code> <code>tools/test_tools.py</code>	The assembler and the golden reference model 4323 cross-validation checks including the 54 hand-verified encodings
<code>tb/*.v</code> , <code>asm/**</code> <code>Report/midterm-proposal.md</code>	The testbenches and test programs, all self-checking The decision table: what was chosen, what was rejected, and why

Numbers derived in this document rather than read from a file (each shown with its derivation where it appears): the cost of the BHT (+216 LUT / +130 FF, from the full/base utilization difference); the flip-flop budget (1052 declared vs 819 implemented); the cycle accounting for `fib` ($4 + 658 + 62 + 176 = 900$); the static program sizes (counted non-zero words in the committed `.hex` images).